Candidate Name: **Binali Assalaarachchi**
BCS Membership No: **995123167**

Supervisor: **Mr. Teran Subasinghe**
BCS Membership No: **990289069**

August 2025

“Student Performance Prediction and Schedule Generation”
Mobile Application

Word Count: **[10,425 words]**

A dissertation submitted in complete fulfilment of the BCS – Chartered Institute of IT’s
[BCS – Professional Graduate Diploma (PGD) in IT – Project, BCS-HEQ]

Abstract

The goal of this project is to develop a mobile application that offers students with individualized support to assist them perform better academically. Based on historical performance, the app forecasts future academic results and generates personalized study plans for enhanced learning.

The system plans to help students better manage their study time and goals by providing features like performance prediction, automatic schedule generation, and a user-friendly interface.

To provide precise predictions of academic performance and individualized study schedules, the system integrates machine learning and mobile app development. It emphasizes on a user-friendly interface for simple interaction. Usability and feature effectiveness were evaluated through user acceptance surveys, which assisted in improving based on actual user input.

The prediction and scheduling features were particularly favored by users, reflecting on the app's simplicity and reliability. This positive feedback demonstrates how well the app supports students' learning objectives and highlights opportunities for future development.

Table of Content

1. Introduction.....6
2. Requirements Analysis and Specification.....7
2.1 Approach to Requirements Gathering.....7
2.1.1 Questionnaires.....7
2.1.2 Review of Current Systems.....8
2.2 Stakeholder Identification and Communication.....9
2.2.1 Engagement.....9
2.2.2 Feedback and Corresponding Requirement.....9
2.3 Comparative Analysis of Existing Systems.....10
2.3.1 Moodle.....10
2.3.2 Khan Academy.....11
2.3.3 OpenAI Academy.....12
2.4 Functional Requirements.....13
2.5 Non-Functional Requirements.....14
2.6 Project Constraints.....15
2.7 Technology and Tools Justification.....16
2.7.1 Programming Language - Python (Django).....16
2.7.2 Machine Learning Framework - Scikit-learn.....17
2.7.3 Mobile Development Framework - Flutter.....18
2.7.4 Database - MySQL.....18
2.7.5 Google Calendar API.....19
2.8 Initial Work Schedule/Timeline.....20
3. Design.....21
3.1 Architectural Design.....21
3.1.1 High-Level Design Overview.....21
3.1.2 Architecture Model.....21
3.1.3 Architecture Diagram.....22
3.1.4 Technologies Mapped to Architecture Layers.....22
3.2 System Flow.....23
3.2.1 Interaction of Major Components.....23
3.2.2 Component Diagram Overview.....23
3.2.3 Subsystem Relationships.....24
3.3 User Interface Design.....25
3.4 Database Design.....27
3.5 Security Design.....29
3.6 Validation Design.....30
4. Implementation.....31
4.1 Tools and Environment Setup.....31
4.2 Implementation of Key Features.....32
4.2.1 Performance Prediction Module.....32
4.2.1.1 Model Training Evidence and Dataset Adaptation.....35
4.2.2 Schedule Generation.....39

4.2.3 User Authentication....41
4.3 Code Structure and Organization....45
4.4 Challenges and Solutions....46
5. Testing....47
5.1 Overview of Testing Approach....47
5.2 Test Plan and Test Cases....48
5.2.1 Login Module....48
5.2.2 Registration Module....49
5.2.3 Prediction and Schedule Generation Module....49
5.2.4 Database Operations....50
5.3 User Acceptance Testing (UAT)....51
5.4 Bug Log and Resolution....56
5.5 Validation....57
5.6 Summary....58
6. Critical Appraisal....59
6.1 Achievements Against Objectives....59
6.2 Challenges Faced....60
6.3 Lessons Learned....61
6.4 Future Improvements....62
6.5 Reflection on Development Process....63
6.5.1 Reflection on Machine Learning Techniques and Data Limitations....63
6.5.2 Code Origin and Adaptation Declaration....64
7. References....65
8. Appendices....66
8.1 Appendix A: Source Files....66
8.2 Appendix B: Source Codes....66
B.1 main.dart....66
B.2 theme.dart....67
B.3 register_screen.dart....67
B.4 login_screen.dart....72
B.5 home_screen.dart....76
B.6 dashboard_screen.dart....78
B.7 train_model.py....79
B.8 app.py....81
8.3 Appendix C: Backend/API Evidence....85
8.4 Appendix D: User Impact....86
8.5 Appendix E: Traceability Table....86
8.6 Appendix F: UAT Feedback Form....87

List of Figures

Figure 3.1.3.1 High-Level System Architecture
Figure 3.2.2.1 Component Diagram
Figure 3.3.1 UI/Wireframe
Figure 3.3.2 Schedule View Screen
Figure 3.4.1 Entity Relationship Diagram
Figure 3.4.2 Logical Table Structure
Figure 4.2.1.1 Prediction endpoint (Simple version)
Figure 4.2.1.2 Prediction endpoint (Extended version)
Figure 4.2.1.3 Model training
Figure 4.2.1.1.1 Import Dataset
Figure 4.2.1.1.2 Integrate Best Model with Backend
Figure 4.2.1.1.3 Prediction Output
Figure 4.2.1.1.4 Confusion Matrix Generation
Figure 4.2.1.1.5 Confusion Matrix
Figure 4.2.2.1 Schedule generation (Client side)
Figure 4.2.2.2 Schedule saving logic (Server side)
Figure 4.2.3.1 User signup
Figure 4.2.3.2 Register Screen Flutter Code
Figure 4.2.3.3 User login and logout
Figure 4.2.3.4 Login Form Layout
Figure 5.3.1 User Demographics
Figure 5.3.2 User Feedback on App Features
Figure 5.3.3 User Feedback on Bugs
Figure 5.3.4 User Feedback on Most Helpful App Feature(s)
Figure 5.3.5 User Feedback on App Navigation and Usage
Figure 5.3.6 User Feedback on Satisfaction
Figure 5.3.7 Improvement Suggestions
Figure 5.3.8 User Feedback on Future App Usage

1. Introduction

In today's stressful academic environment, students are frequently overwhelmed with plenty of assignments, lectures, and extracurricular activities. A lot of students find it difficult to maintain regular study routines, which leads to stress and poor performance. This emphasizes the necessity of digital resources that help students better plan their study schedules and fully understand their academic progress.

The goal of this project is to deliver students with intelligent support in order to improve their study habits and encourage academic success. Machine learning and mobile app development are integrated to create a system that not only tracks user input and academic data but also offers insightful analysis and practical recommendations.

The objectives of the system are to:

- Develop a user-friendly mobile application that facilitates log in/sign up, academic data input, and user interaction.
- Develop a performance prediction model utilizing machine learning techniques.
- Implement a scheduling function that creates study schedules based on user input.
- Through user acceptance testing, assess the application's efficiency and implement necessary system improvements in response to feedback from users.

Key software development practices are also addressed in the project, with Agile methodology being utilized to guarantee continuous feedback, design flexibility, and incremental development. Flutter is used to develop the frontend, and Python(Django) is used for backend services. Additionally, a MySQL database is integrated into the system to securely store user information and prediction outcomes.

This report is structured into several sections. The system requirements are outlined in Section 2, including both functional and non-functional requirements. The system design is presented in Section 3, which also covers the database structure, user interface designing, and overall architecture. The implementation procedure is outlined in Section 4, along with the technologies utilized, model integration, and important backend components. The testing phase is explained in Section 5, which also includes the results from user acceptance and functional testing. A critical appraisal is presented in Section 6, which covers the project's accomplishments, possible future enhancements, and opinions on the development process.

2. Requirements Analysis and Specification

The key requirements for the system are outlined in this section. It consists of both functional and non-functional requirements that are based on project goals and user requirements.

2.1 Approach to Requirements Gathering

The successful development of any software system, especially one that aims to offer individualized support based on user-specific data, depends on effective gathering of requirements. For this project, questionnaires and reviews of existing systems were the two primary approaches used for requirements gathering. These techniques were chosen because they are appropriate for understanding user requirements, expectations, and current solutions in the field of academic performance monitoring.

2.1.1 Questionnaires

Questionnaires were selected as a primary method for gathering requirements because they can reach a large audience, especially students, who are the primary users of this system. Since the objective of this project is to assist students by predicting academic performance, it was important to understand students' present study habits and challenges they face.

Through well-structured and brief questions, data was collected on:

- Study techniques.
- Frequency of studying.
- Common distractions for lower academic performance.
- Preference for study reminders.
- Willingness to share academic information such as grades or attendance.

This approach was chosen because

- It was possible to gather both quantitative and qualitative data.
- It ensured confidentiality, which motivated students to give honest responses.
- It was both cost-effective and time-efficient.

2.1.2 Review of Current Systems

Analyzing existing systems such as Learning Management Systems (LMS), academic dashboards, and study planner apps made it easier to determine the strengths and weaknesses of potential alternatives.

This offered insightful information about

- The degree of personalization currently available.
- The standard design of user interfaces in educational tools.
- The kinds of data visualizations that students find useful.

To gain a better understanding of how current educational platforms assist students in enhancing their academic performance, systems including Moodle, Khan Academy, and OpenAI Academy were reviewed.

This approach was suitable because

- It helped to concentrate on adding value through predictive analytics rather than having to start from scratch.
- It led to the decisions on what features to add or enhance.

Following the completion of Questionnaires and Review of Current Systems, publicly available datasets from Kaggle were added to further strengthen the project. These datasets include historical academic data, which provides a strong basis for building predictive models and creating customized study plans. (*Student Exam Performance Prediction*, 2024)

2.2 Stakeholder Identification and Communication

This project's core stakeholder is students since they will be the primary users of the application. Understanding their study habits, challenges, and requirements is important to determine the features of the system and to ensure it offers value in assisting them in managing their academic performance.

2.2.1 Engagement

An online questionnaire was shared among students via social media channels. This approach guaranteed broad involvement and enabled students to offer anonymous feedback. The questionnaire focused on areas such as frequency of studying, study techniques, common distractions, and preference for study reminders.

Collection and processing of sensitive personal data such as grades or attendance was carried out with explicit consent and adherence to the privacy policies.

2.2.2 Feedback and Corresponding Requirement

Feedback from students	**Corresponding Requirement**
A lot of students preferred study schedules catered to their unique learning styles.	Provide a tool that allows users to personalize study plans based on their goals and habits.
Students preferred frequent reminders in order to maintain consistency in their study routines.	Implement automated study notifications and reminders.
Many students showed interest in performance tracking and receiving notifications when performance declines.	Create a module for predictive analytics to forecast academic performance.
A significant number of students mentioned digital distractions as a challenge.	Offer focus-oriented features and simple scheduling tools.
Concerns were raised over the sharing of academic data.	Ensure privacy regulations are strictly followed and all personal data are kept anonymous.

Table 2.1 Feedbacks and corresponding requirements

2.3 Comparative Analysis of Existing Systems

Three well-known platforms; Moodle, Khan Academy, and OpenAI Academy were reviewed in order to get a better understanding of the current state of educational support tools. Every system was examined according to its primary function, strengths and weaknesses, and applicability to the proposed system.

2.3.1 Moodle

Moodle is a widely used open-source learning platform designed to support educators in creating customized learning environments *(Moodle, 2025).*

Strengths:

- Supports a variety of academic tools such as assignments, discussion boards and quizzes.
- Enables instructors to grade and monitor student progress.
- Highly flexible and compatible with extensions.

Weaknesses:

- Lacks advanced student recommendations or built-in predictive analytics.
- Available features for study planning are limited and not customizable.
- For the students looking for a more modern and user-friendly experience, the UI may be complicated and outdated.

Gaps Identified:

- Automatic creation of study plans based on individual learning styles is not available.
- No real-time alerts for declines in performance.

Value Added:

The proposed system offers customized study plans and performance forecasting based on machine learning, which Moodle lacks since it mainly focuses on proactive, student-centered academic support.

2.3.2 Khan Academy

Khan Academy provides freely accessible, high-quality educational videos and exercises but lacks personalized study planning *(Khan Academy, 2025)*.

Strengths:

- Proficient tracking to demonstrate a student's level of understanding.
- Engaging and high-quality educational materials.
- Free of charge and globally accessible.

Weaknesses:

- Study reminders are not available.
- Ability to customize is limited.
- Lacks predictive analytics.

Gaps Identified:

- Does not provide study plans tailored to students' learning styles.
- It lacks a feature to notify students when there is a risk for failure.

Value Added:

The proposed system fills these gaps by providing data-driven alerts and automated study reminders.

2.3.3 OpenAI Academy

OpenAI Academy offers modern, AI-based learning tools mainly in the tech domain *(OpenAI, 2025).*

Strengths:

- Uses AI to provide insightful feedback for code-related tasks.
- Provides an interactive learning environment.
- Modern, responsive user interface.
- Promotes real-world, hands-on learning.

Weaknesses:

- The focus is on technical subjects.
- No performance analytics tools.
- Adequate study aids and academic planning tools are not available.

Gaps Identified:

- Personalized study planning is not available.
- It is not intended for general student performance management.

Value Added:

Although OpenAI Academy excels at AI learning, the proposed system expands this by providing personalized study schedules and predictive analytics, all of which are particularly helpful for students studying subjects other than technology.

2.4 Functional Requirements

This section defines the core functions that the proposed system must complete to satisfy user needs. These features will allow students to receive personalized study support, performance prediction, and study reminders.

User Features

- The system must allow users to create and manage accounts on the system.
- The system must allow users to enter basic academic information such as study hours, subjects, and upcoming tests.
- The system must be able to provide personalized academic support to users based on their input data.
- The system must allow users to update their objectives and study preferences.
- The system must deliver study reminders and notifications based on the user's schedule.
- The system must send real-time notifications if there is a decline in performance.
- The system must display predicted analysis results of academic performance.
- The system must allow users to monitor their progress.

System Features

- The system shall generate customized study schedules using machine learning algorithms.
- The system shall analyze historical data to predict academic performance.
- The system shall automatically update the study plan based on the user's updated data.
- The system shall guarantee that user data is stored and processed in compliance with privacy regulations.

2.5 Non-Functional Requirements

This section outlines the system's operations on performance, usability, accessibility, security, and scalability. These requirements guarantee that the system provides a reliable and seamless user experience.

Performance:

- The system should respond to user interactions, such as loading the dashboard and generating study plans, within 2-5 seconds.

Usability:

- The system should provide a simple and easy-to-use interface that supports students without much technical knowledge.
- The system should clearly display error messages.
- The system should support a variety of screen sizes.

Accessibility:

- The system must be accessible on smartphones, laptops, and tablets.
- The system should support assistive technologies like keyboard navigation and screen readers to be accessible for people with disabilities.

Security:

- The system must use encryption for secure data storage.
- The system should use role-based access control and authentication to limit access to sensitive information.
- The system should track and keep records of suspicious login attempts.

Scalability:

- The system must be able to grow with an increasing number of users.

2.6 Project Constraints

Several limitations had to be considered during the development of this project.

- Time Constraints - Due to the limitation of the time period to complete the project within 8 weeks, priority will be given to the development of crucial elements, including performance prediction, study plan creation, and real-time alerts.

- Technical Skills and Resources - The system will be developed based on the proficiency level of a university student with access to common academic resources, and advanced functionalities will be scoped to fit the available resources and tools.

- Platform and Browser Compatibility - The system needs to be accessible across major platforms (such as Windows, Android, and iOS), and that makes the UI design and testing a bit complex.

- Data Availability and Licensing - Since the project uses Kaggle datasets, which are publicly available, it needs to be ensured that the data is open-source and does not violate any privacy laws.

- Budget and Hosting Limitations - Only free and open-source platforms and tools will be utilized due to the lack of funding.

2.7 Technology and Tools Justification

This section outlines the tools, technologies, and frameworks that will be utilized for the development of this project along with how they will be utilized and the reason for choosing each one.

2.7.1 Programming Language - Python (Django)

Python will be utilized to construct the logic for machine learning models, and Django offers the framework required to manage web application features like routing, views, and managing requests from mobile and web interfaces. *(Django, 2024)*.

- Familiarity and Learning Curve: Python is well-known for being user-friendly and effective, and Django offers simple syntax and built-in features.

- Suitability for the Application: Python's extensive libraries, including Scikit-learn, are perfect for analyzing data and implementing machine learning models. Django is a good fit for managing the web application's backend infrastructure.

- Compatibility with Hosting/Server: Python and Django are compatible with well-known cloud computing platforms like Google Cloud and ensure scalability and easy deployment.

- Community Support and Documentation: Python and Django have large communities and extensive documentation, offering multiple tools for troubleshooting and development.

2.7.2 Machine Learning Framework - Scikit-learn

To analyze historical data and forecast student performance, various machine learning algorithms, including regression and classification models, will be implemented using Scikit-learn. *(Scikit-learn, 2024)*.

- Familiarity and Learning Curve: Scikit-learn is easy to use and well documented, offering many examples and tutorials for developers of all skill levels.

- Suitability for the Application: Scikit-learn provides a large collection of machine learning algorithms, model evaluation, and data preprocessing tools for forecasting student performance based on historical data.

- Compatibility with Hosting/Server: Scikit-learn simply integrates with Python-based frameworks such as Django, allowing the machine learning model to be hosted and accessed through the backend of the application.

- Community Support and Documentation: Scikit-learn provides a wide range of tools and assistance for developing and debugging models.

The table compares Scikit-learn with alternatives like TensorFlow, and PyTorch to determine its suitability for this project.

Framework	**Use Case**	**Ease of Use**	**Strengths**	**Limitations**	**Suitability**
Scikit-learn	Classical ML	Easy	Simple Good documentation	No deep learning	Best for tabular data and classical ML
TensorFlow	Deep Learning	Moderate	Scalable Efficient	Complex	Best for deep learning
PyTorch	Deep Learning	Moderate	Flexible Research-oriented	Complex setup	Best for deep learning research

Table 2.3 Comparison of ML Frameworks

Scikit-learn was chosen due to its simplicity, robust documentation, and suitability for classical machine learning workflows, especially on tabular data *(Pedregosa et al., 2011; scikit-learn Developers, 2025)*.

2.7.3 Mobile Development Framework - Flutter

Flutter will be used to create a mobile application that supports both Android and iOS and provides a better user experience. *(Flutter, 2024)*.

- Familiarity and Learning Curve: Since a wide range of resources and tutorials are available, Flutter is relatively easy to learn.

- Suitability for the Application: Flutter guarantees smooth operation on both Android and iOS, which expands the application's user base.

- Compatibility with Hosting/Server: Flutter can easily communicate data between the mobile application and the backend.

- Community Support and Documentation: Flutter provides lots of resources and documentation for developers.

2.7.4 Database - MySQL

MySQL will be the relational database management system (RDBMS) used to store user data. This guarantees secure data handling and makes querying and retrieval of information efficient.

- Familiarity and Learning Curve: MySQL is a popular and well-known database system that is simple to use and integrate into the project.

- Suitability for the Application: MySQL is ideal for handling structured data since it allows for complex querying as well as the efficient storage and retrieval of student performance data.

- Compatibility with Hosting/Server: MySQL works well with Django and is compatible with major cloud hosting platforms.

- Community Support and Documentation: MySQL is documented and supported by a large community, making it simple to obtain resources for optimization and troubleshooting.

The table below evaluates the suitability of MySQL and MongoDB for this project .

Database	**Data type**	**Strengths**	**Limitations**	**Suitability**
MySQL	Structured	Comply with ACID principles	Rigid schema	Best for structured data
MongoDB	Unstructured	Flexible schema	Weak joins	Not ideal for this project

Table 2.4 Comparison of Databases

MySQL was selected due to its strong ACID compliance, structured query capabilities, and mature ecosystem *(Oracle, 2025)*.

2.7.5 Google Calendar API

The Google Calendar API will be used to manage notifications and reminders.

- Familiarity and Learning Curve: Integrating Google Calendar API into the application is simple since it has Python client libraries available.

- Suitability for the Application: Google Calendar API integrates scheduling and reminders into the application, allowing students to receive real-time notifications and manage their study schedules accordingly.

- Compatibility with Hosting/Server: Due to the API's smooth integration with Django, the application can synchronize events and send notifications to users' Google Calendar accounts.

- Community Support and Documentation: This guarantees easy integration and troubleshooting through detailed documentation and a large developer community.

2.8 Initial Work Schedule/Timeline

This table illustrates the Agile 8-week project schedule, which starts on 7th April, 2025, and ends on 2nd June, 2025. This is structured into 6 sprints, each lasting one week, and covers all the important stages, including requirements gathering, design, implementation, testing, and documentation.

Sprint	**Time Duration**	**Planned Activities**
Sprint 1- Requirements Gathering	4/7/2025 - 16/7/2025	● Identify project requirements. ● Collect data via questionnaire, review of existing systems, and publicly available dataset from Kaggle. ● Initial data exploration and analysis.
Sprint 2- Data Preprocessing	4/15/2025 - 4/21/2025	● Clean missing and inconsistent data. ● Feature engineering and selection. ● Prepare the dataset for modeling.
Sprint 3- Model Development and Training	4/22/2025 - 5/2/2025	● Select and train machine learning algorithms. ● Evaluate model performance.
Sprint 4- Testing and Evaluation	5/3/2025 - 3/10/2025	● Test the model with new data. ● Compare with baseline performance. ● Document results.
Sprint 5- Development	5/11/2025 - 5/25/2025	● Design the UI. ● Integrate the predictive model. ● Add the personalized study schedule feature.
Sprint 6- Final Testing, Deployment and Documentation	5/26/2025 - 6/2/2025	● User testing and feedback collection. ● Finalize deployment. ● Prepare the final project documentation.

Table 2.5 Agile based project timeline

3. Design

The planning of the structure and workflow of the mobile application was the main focus of this phase. It included the architecture design, system flow design, user interface design, database design, security design, and validation design. This phase ensured that the functionality of the app aligned with the objectives of the project.

3.1 Architectural Design

This section highlights the overall structure of the system. It shows how different components interact with each other to ensure smooth functioning of the system.

3.1.1 High-Level Design Overview

The system used a client-server architecture in which a backend server communicated with the mobile client to retrieve user-related data, predictions, and customized schedules. This architecture clearly separated the user interface from the underlying processing mechanism, encouraging scalability and maintainability. Students were able to review performance trends, get reminders, and effectively manage their study schedules through the app's user-friendly interface.

3.1.2 Architecture Model

The Model-View-Controller (MVC) architecture framework was applied. The View was the frontend developed using Flutter that manages data presentation and user interface. The Controller controls how the frontend and backend interact, including how requests are handled and validated. The Model illustrated the machine learning prediction model and database layer, which stored and processed academic data and performance records. This separation provided an organized code and made it easier for each module to be developed and tested independently.

3.1.3 Architecture Diagram

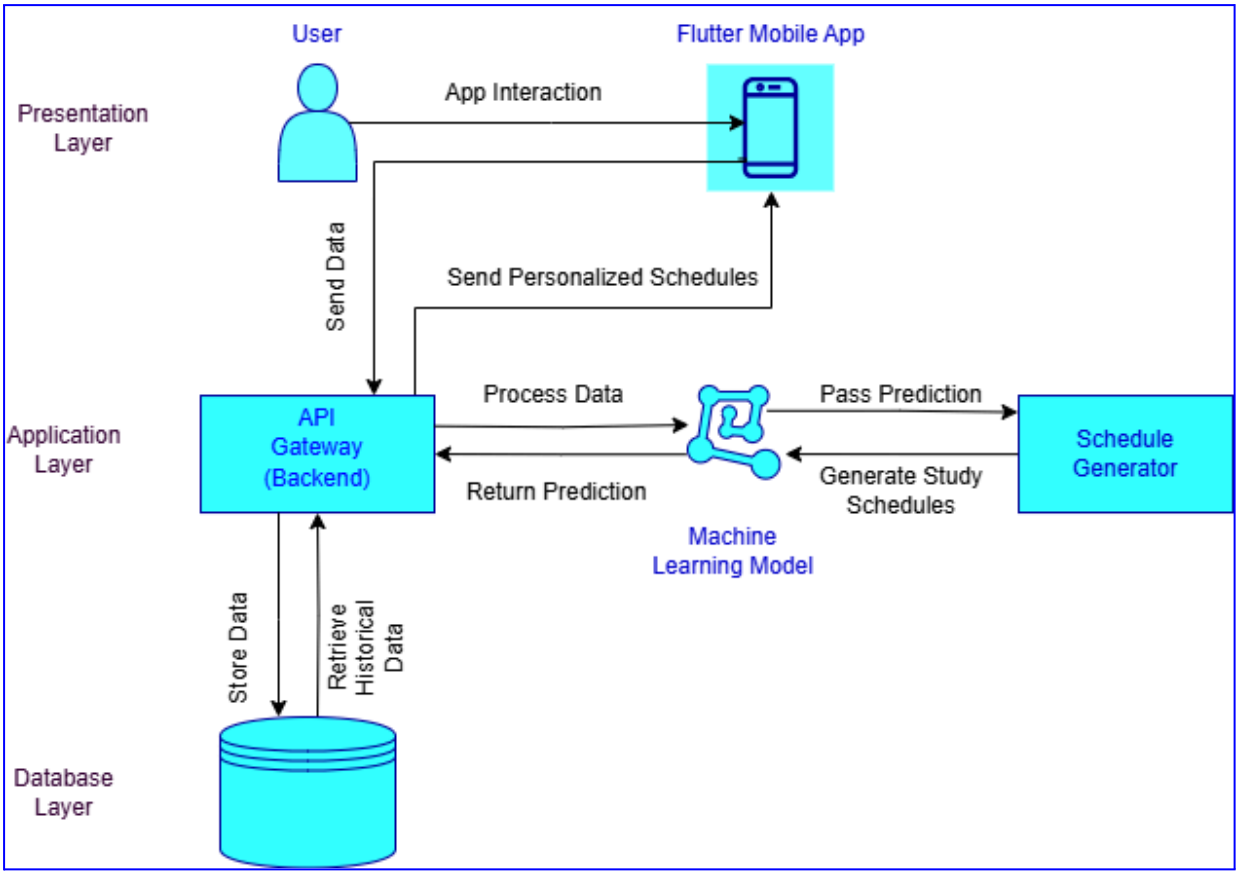


Figure 3.1.3.1 High-Level System Architecture

Figure 3.1.3.1 displays the system architecture for the student academic prediction and scheduling mobile application. This followed a three-layer architecture: Presentation Layer, Application Layer, and Database Layer. The Flutter mobile application was connected to the backend and interacted with users through an API Gateway. For performance prediction, the data was processed by the backend and sent to the machine learning model. The Schedule Generator generated customized study schedules based on the predictions. The student data and generated schedules were stored and retrieved from the database.
This diagram was created using Draw.io *(draw.io - free flowchart maker and diagrams online).*

3.1.4 Technologies Mapped to Architecture Layers

Flutter was used to develop the frontend, which supported both Android and iOS platforms. Students were able to interact with their academic information and personalized schedules through a user-friendly interface. Python was used to create the backend, which managed schedule creation, prediction logic, and data processing. The machine learning model, which was trained on Kaggle datasets, examined historical data to predict performance decline and suggest customized schedules. MySQL was used in the database layer, which allowed data consistency, effective querying, and simple integration with backend logic.

3.2 System Flow

This section highlights how data moves through the system, showing the interactions between app, backend, ML model, schedule generator, and database.

3.2.1 Interaction of Major Components

The Flutter Mobile App (User Interface), Backend API, Machine Learning Prediction Engine, Schedule Generator, and MySQL Database were the main components that made up the system. The students were able to input data and receive customized study schedules using the mobile app. The backend, which served as the central controller of this system, received this data. It coordinated and handled requests between the ML model, schedule logic, and database.

3.2.2 Component Diagram Overview

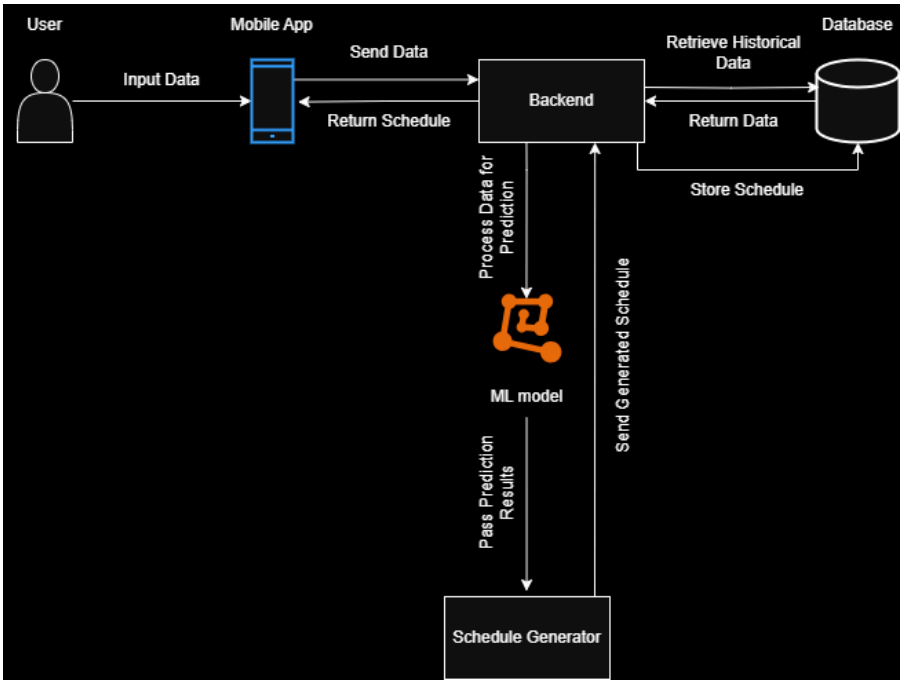


Figure 3.2.2.1 Component Diagram

Figure 3.2.2.1 illustrates the system workflow, where modules interact with each other. The backend processed the academic data submitted by the user and retrieved historical data from the database. Then this data was sent to the ML model, which generated performance predictions. The schedule generator developed a study plan based on these predictions. The user was able to access the final schedule after it had been delivered back to the mobile app via the backend.

3.2.3 Subsystem Relationships

Each subsystem had a specific function and interacted with other subsystems through predefined interfaces. HTTP requests were used for communication between the user interface and the backend. The ML model, schedule logic, and database were all connected by the backend. The ML model and schedule generator were strongly integrated to guarantee that predictions are instantly converted into realistic study plans. The database solely communicated with the backend, ensuring data integrity and centralized access control.

3.3 User Interface Design

The User Interface (UI) was designed to be minimal and responsive, ensuring ease of use on both Android and iOS platforms. This UI, which was developed with a mobile-first approach using Flutter, emphasizes simplicity, clarity, and fast access to essential elements like academic monitoring, schedule viewing, and user profile management.

As shown in Figure 3.3.1 below, the login screen has a ‘Sign Up’ option for new users in addition to the email and password fields for ‘Login.’ Large input fields and clear spacing were maintained to facilitate ease of use on smaller screens. Strong contrast for readability was ensured by combining purple variants with black and white for text to create a modern and user-friendly interface. In line with the app’s academic focus and mobile-first approach, the minimalist design assisted in reducing distractions.

The Dashboard Design includes key features like predicted performance, schedule overview, and progress report. Academic health was efficiently expressed using icons and colored indications (e.g., green for high performance, red for deterioration). The card-based structure of the layout was familiar and simple to use, especially on smaller screens.

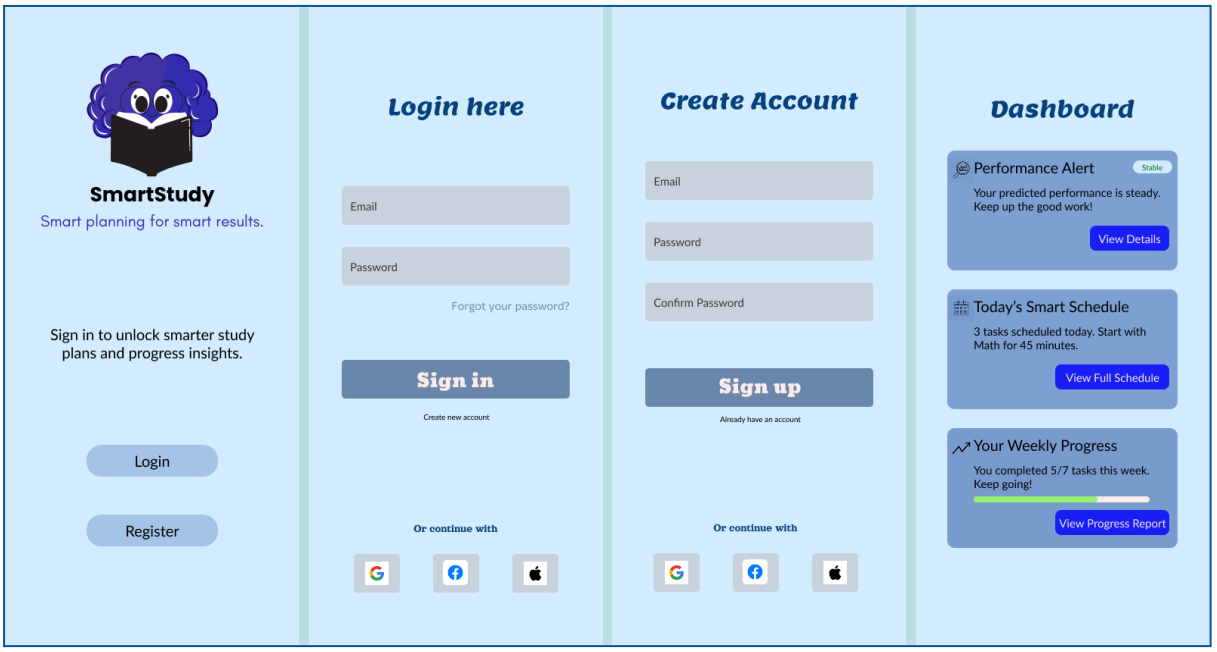


Figure 3.3.1 UI/Wireframe

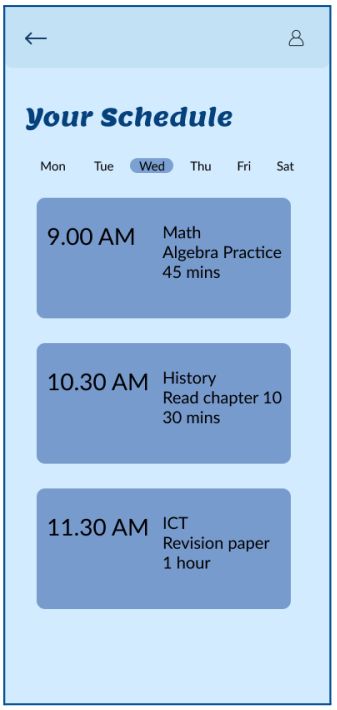


Figure 3.3.2 Schedule View Screen

A Schedule View Screen was another feature of the interface, which displayed the created study plan in a calendar-style arrangement. As a result, students found it easier to comprehend daily objectives and manage their time. To increase interaction speed, touch gestures like tapping tasks and swiping between days were available.

Wireframes and mockups were initially created using Figma, allowing for iterative improvements based on user feedback. (*Figma—The Collaborative Interface Design Tool*)

A major consideration during the design process was mobile responsiveness. Every layout effortlessly adjusts to different screen sizes and orientations. Consistent spacing, alignment, and interactivity across devices were guaranteed when Flutter's adaptable layout widgets were used.

Overall, the design combined excellent usability with aesthetic simplicity to provide students with a simple and customized experience that helps them comprehend and enhance their academic performance.

3.4 Database Design

The database included tables like User, Subjects, Tasks, Schedules, and Performance_Statistics, all of which are essential for organizing student-related information and enabling the smart features of the application. Features like performance prediction, customized study schedule generation, and task monitoring were supported by the manner in which these entities were organized.

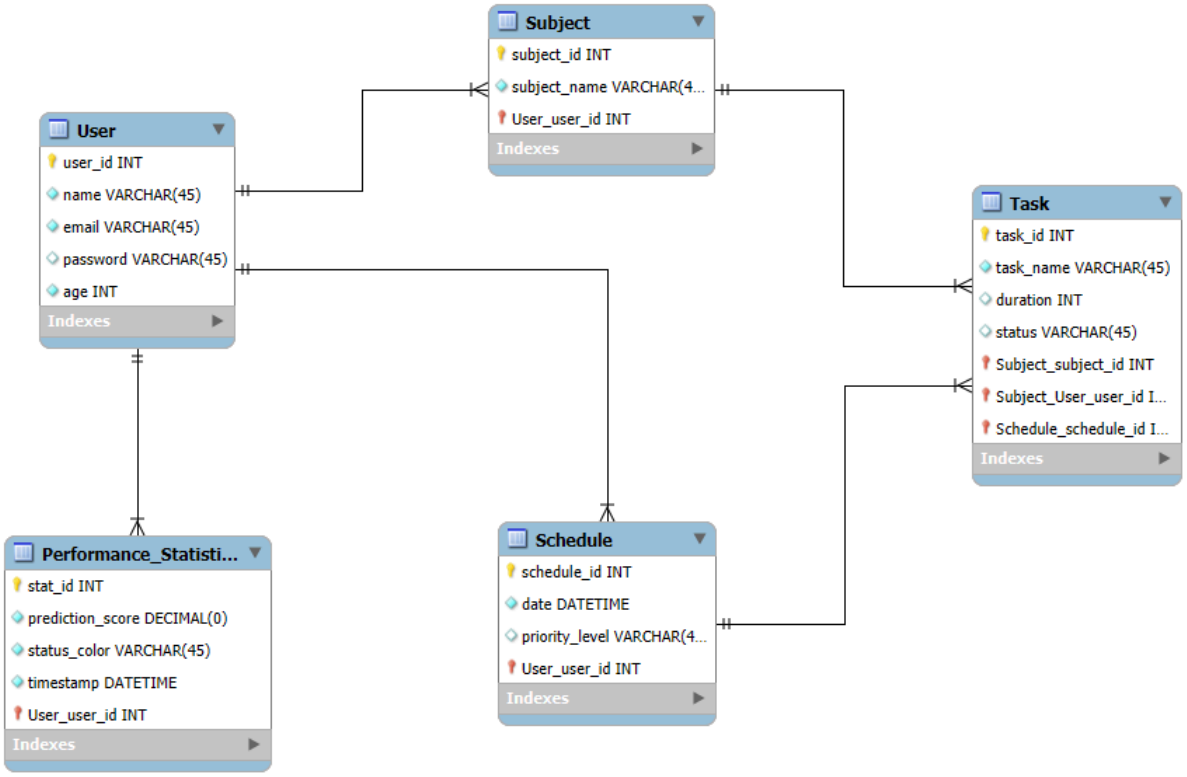


Figure 3.4.1 Entity Relationship Diagram

Figure 3.4.1 illustrates the ERD that models the structure of this system's database. It consists of five main entities: Users, Subjects, Tasks, Schedules, and Performance_Statistics, each supporting the core functionalities, including user management, performance prediction, schedule generation, and task monitoring.

The Users table stored vital information about each student, and the Subjects table linked directly with users, enabling personalized subject management. To keep track of learning activities on specific days, the Tasks table was linked to both Subjects and Schedules. Each user's daily plans were represented in the Schedules table, and the system's performance model generated prediction data, which was stored in the Performance_Statistics table.

The database had been normalized to the Third Normal Form (3NF). This ensured that repetitive data was removed, primary key dependence was guaranteed, and efficiency was increased. For instance, normalization ensured that subjects were stored once and accessed via foreign keys, avoiding the need to duplicate subject names across tasks.

This well-structured and normalized schema formed the foundation for a responsive and smart student dashboard system, which guarantees scalability, ease of maintenance, and consistent performance.

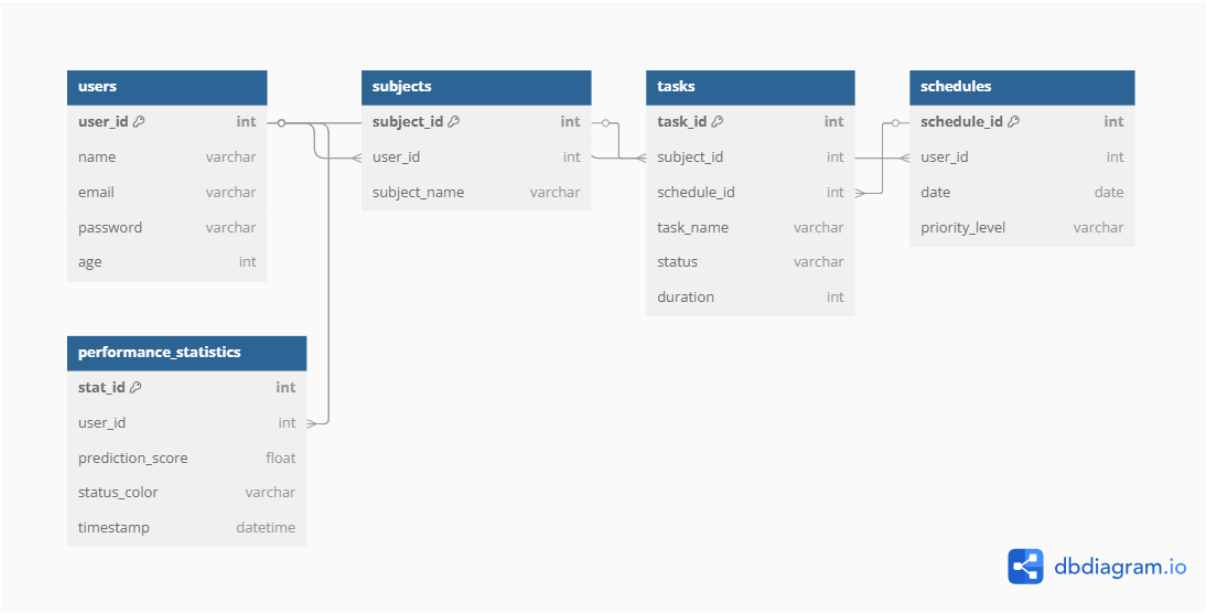


Figure 3.4.2 Logical Table Structure

Figure 3.4.2 illustrates the fundamental database schema, which consists of five tables.

- Users table saves students' information such as id, name, age, email, and password.
- Subjects table provides individualized subject tracking by connecting with the Users table.
- Tasks table contains tasks related to particular subjects and schedules, along with task status and duration.
- Schedules table keeps track of the schedules of each user with dates and priority levels.
- Performance_Statistics table tracks user performance over time by recording time and performance prediction score.

3.5 Security Design

Multiple security levels were implemented by the system to guarantee the safety and privacy of user data. Prior to being saved in the database, passwords were hashed using industry-standard techniques like bcrypt. This protected private credentials even in the event of unauthorized access.

To manage user sessions, the system utilized secure session tokens, which were invalidated after an established inactivity period or after a user logged out. As a result, the probability of session hijacking was reduced. Since HTTPS secured all data transmission, credentials and other sensitive data could not be intercepted.

Access control was implemented through role-based permissions. For example, administrators may have more authority to manage the system, while the students can only view their schedules and performance records. This restricted the exposure to data based on user roles.

Further security features included restricted API endpoints, frequent authentication token refreshes for extended sessions, and input validation to avoid SQL injection.

All users were guaranteed a secure and reliable experience since only authorized users were able to access the system's essential features, and any unauthorized attempts to access it were recorded and blocked.

3.6 Validation Design

Form validation ensured secure and accurate data input by combining frontend and backend procedures. Libraries like Formik were used on the frontend to verify needed inputs, password strength, and email format before submission. This enhanced user experience by giving immediate feedback.

The system used bespoke algorithms to perform server-side validation on the backend, rechecking all inputs and preventing browser manipulation to bypass the system. Inputs like name, email, password, task duration, and date formats were checked by the system to make sure they adhered to predetermined guidelines.

For instance, when a user registers, the system verifies that the email address is legitimate and hasn’t been used before and that the password satisfies requirements for complexity (e.g., minimum 8 characters, including symbols). The form displayed error messages in real time.

4. Implementation

This stage focused on converting the design into a working application using chosen tools and technologies. It included setting up the development environment, implementing key features through backend code, developing frontend code, and integrating the machine learning model.

4.1 Tools and Environment Setup

The implementation was carried out using a collection of integrated tools that facilitated both frontend and backend. Visual Studio Code (VS Code) was used since it supports multiple languages and extensions that simplify Django and Flutter development. Its built-in terminal made it easy to navigate between project files and smoothly execute commands.

To build the backend using Django, Python 3.13 was installed. The Flutter mobile app's communication APIs were exposed using the Django REST framework. Additionally, the trained machine learning model was loaded and used through joblib and scikit-learn.

The database system, MySQL, was controlled locally via XAMPP. The 'mysqlclient' package was used to connect MySQL to the Django backend. Postman made it possible to verify the API endpoints before connecting to the mobile frontend, which aided in testing the REST APIs as demonstrated throughout the development.

For version control, Git and GitHub were used. Git was used to log and commit all changes, and GitHub was used to host the project repository in order to facilitate collaboration and backup.

Setting up the environment for local development involved configuring Django settings to connect to the MySQL database and using pip to install necessary Python packages. Flutter was used to construct the mobile application separately, and REST API calls were used to connect it to the backend.

4.2 Implementation of Key Features

This section focuses on the key features of the mobile application, grouped according to the user role (Student). This explains each feature's implementation details, challenges encountered, and its functionality within the overall system.
Below is an overview of the major parts of the codebase. Full source code is included in **Appendix A**.

4.2.1 Performance Prediction Module

A key part of the backend system is the performance prediction module, which evaluates a student's likelihood of passing based on a variety of academic and personal criteria. Python's Flask framework was used to develop this feature, which was then combined with a machine learning model that was trained using Scikit-learn and deployed via the 'joblib' serialization technique.

Students can submit relevant academic data through the 'predict-performance' API endpoint, and a machine learning model located on the backend processes it. This endpoint manages POST requests that contain JSON data with input fields such as,

- Simplified form: hours studied and previous grades
- Extended form: detailed list including gender, ethnicity, scores in different subjects, etc.

```python
@app.route('/predict-performance', methods=['POST'])
def predict_performance():
    print("POST request received at /predict-performance")
    data = request.get_json()
    print("Data received:", data)

    hours_studied = data.get('hours_studied')
    previous_grade = data.get('previous_grade')

    if hours_studied is None or previous_grade is None:
        return jsonify({'error': 'Missing data for prediction'}), 400

    prediction = model.predict([[hours_studied, previous_grade]])

    prediction_result = int(prediction[0])

    return jsonify({'prediction': prediction_result})
```

Figure 4.2.1.1 Prediction endpoint (Simple version)

In the simplified form of the prediction endpoint, only two fields, 'hours_studied' and 'previous_grade,' are accepted. The machine learning model accepts them and returns a binary classification that indicates the likelihood of the student passing.

```python
def predict_performance():
    data = request.get_json()

    prediction = model.predict(features)

    try:
        features = [
            data['gender'], data['race_ethnicity'], data['parental_level_of_education'],
            data['lunch'], data['test_preparation_course'], data['math_score'],
            data['reading_score'], data['writing_score']
        ]
    except KeyError:
        return jsonify({'error': 'Missing one or more required fields'}), 400

    try:
        features_array = np.array(features).reshape(1, -1)
        prediction = model.predict(features_array)[0]
        return jsonify({'prediction': int(prediction)})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

def predict_performance():
    if 'user_id' not in session:
        return jsonify({"error": "You must be logged in to access this route"}), 401

    data = request.get_json()
```

Figure 4.2.1.2 Prediction endpoint (Extended version)

On the other hand, the extended form of the prediction endpoint which requires more comprehensive data, including gender, ethnicity, scores in different subjects, etc. The API is made more reliable and user-friendly by including error handling to detect missing fields or invalid input types.

Both of these versions return the prediction result as a JSON object, which is displayed to the users through the frontend.

How it works:

1. **Input Handling:** The endpoint uses 'request.get_json()' to read incoming JSON data.
2. **Validation:** It checks for the necessary keys. It returns a 400 HTTP status along with a relevant error message if any input is missing.
3. **Model Prediction:** The inputs are passed to an independently trained Decision Tree model, which is saved as 'student_model.pkl.'
4. **Response Generation:** A prediction value (0 for fail, 1 for pass) is returned as the outcome.

A Kaggle dataset of students' study hours and previous exam scores was used to train the prediction model. Decision Tree was chosen for implementation since its accuracy of 99% outperformed the other two models, Logistic Regression and Support Vector Machine (SVM).

```python
X = data[['Study Hours','Previous Exam Score']]  #Prediction features
y = data['Pass/Fail']                            #Target label

#Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

results = {}

#Train Logistic Regression model
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
log_preds = log_model.predict(X_test)
log_acc = accuracy_score(y_test, log_preds)
results['Logistic Regression'] = (log_acc, log_model)

#Train SVM model
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)
svm_preds = svm_model.predict(X_test)
svm_acc = accuracy_score(y_test, svm_preds)
results['SVM'] = (svm_acc, svm_model)

#Train Decision Tree Classifier model
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X_train, y_train)
tree_preds = tree_model.predict(X_test)
tree_acc = accuracy_score(y_test, tree_preds)
results['Decision Tree'] = (tree_acc, tree_model)
```

Figure 4.2.1.3 Model training

4.2.1.1 Model Training Evidence and Dataset Adaptation

A simplified dataset from a Kaggle source was utilized to train the machine learning model. Students' study hours, previous exam results, and a binary label indicating pass or fail were present in the dataset. Only two features, Study Hours and Previous Exam Score were chosen for the simplified model after the dataset was loaded using Pandas. These were selected to simplify initial implementation as they have a strong correlation with performance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, f1_score
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import numpy as np
import joblib

#Load dataset
data = pd.read_csv(r'C:\Users\ASUS\Downloads\student_exam_data_new.csv')
```

Figure 4.2.1.1.1 Import Dataset

The train_test_split() function in Scikit-learn was used to split the dataset into training and testing sets in an 80:20 ratio once it had been preprocessed using the Pandas library. Three classification models were trained and assessed.
The accuracy score metric on the testing set was used to assess each model once it had been trained using the training set.

Model	Accuracy
Logistic Regression	0.8600
Support Vector Machine(SVM)	0.8600
Decision Tree Classifier	0.9900

Table 4.1 Accuracy Score Metric

With an accuracy of 99%, the Decision Tree model outperformed the other models and was selected as the final model. To integrate it with the Flask backend, it was serialized and saved using joblib.

```python
for name, (acc, _) in results.items():
    print(f"{name} Accuracy: {acc:.4f}")

#Identify and save best model based on accuracy
best_model_name = max(results, key=lambda x: results[x][0])
best_model = results[best_model_name][1]
joblib.dump(best_model, 'student_model.pkl')
print(f"\nBest model: {best_model_name} saved as 'student_model.pkl'")

#F1-score and Classifiaction report
y_pred = best_model.predict(X_test)
print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
print(classification_report(y_test, y_pred))
```

Figure 4.2.1.1.2 Integrate Best Model with Backend

Metrics like F1-score were considered in order to assess the model in addition to accuracy due to its capacity to strike a balance between precision and recall, especially in classification tasks. Even though the F1-score has been determined to be a valuable metric for this project, comprehensive metric computations and visualizations (such as a confusion matrix) are yet to be implemented and scheduled for the future.

```
Logistic Regression Accuracy: 0.8600
SVM Accuracy: 0.8600
Decision Tree Accuracy: 0.9900

Best model: Decision Tree saved as 'student_model.pkl'
F1 Score: 0.9899683371547112
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        64
           1       1.00      0.97      0.99        36

    accuracy                           0.99       100
   macro avg       0.99      0.99      0.99       100
weighted avg       0.99      0.99      0.99       100
```

Figure 4.2.1.1.3 Prediction Output

To assess the chosen Decision Tree model's classification performance, a confusion matrix was generated. The following are the outcomes:

```python
#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```

Figure 4.2.1.1.4 Confusion Matrix Generation

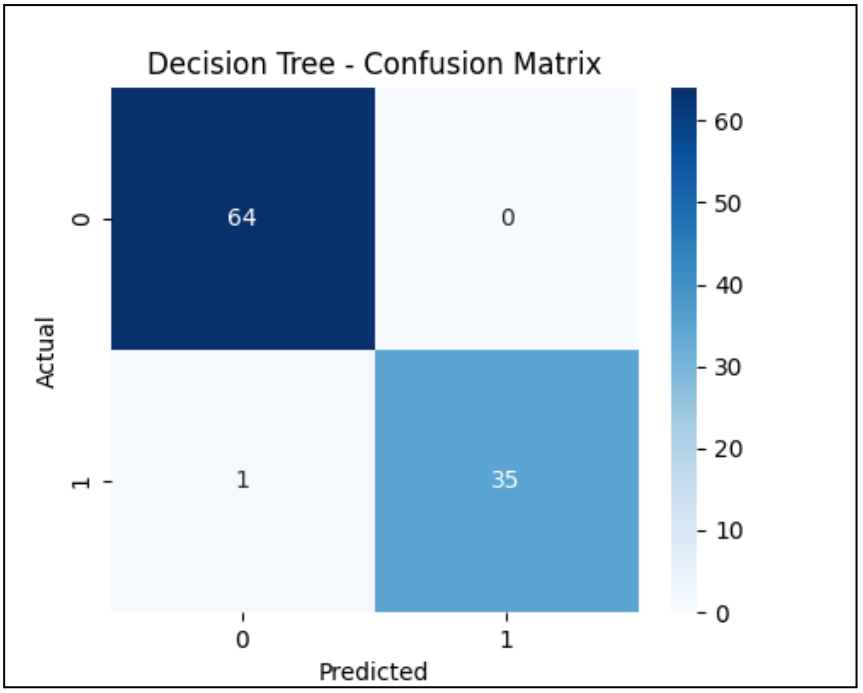


Figure 4.2.1.1.5 Confusion Matrix

- True Negatives (TN): 64 instances in which failing students were correctly identified as failing.
- True Positives (TP): 35 instances in which passing students were correctly identified as pass.
- False Negatives (FN): One instance in which a student who passed was mistakenly labeled as failing.
- False Positives (FP): 0 instances in which a failing student was mistakenly labeled as pass.

With incredibly low misclassification rates, the model's near-perfect diagonal dominance suggests exceptional predictive ability. The test set's only misclassification is represented by the single false negative, and the model's long-term stability is further supported by the lack of false positives.

Challenges faced:

- Despite the decision tree model's 99% accuracy, there were concerns about possible overfitting, particularly considering the limited number of inputs in the basic implementation. To ensure effective model generalization, cross-validation and more testing on unobserved data are planned for future enhancements.

- Another main challenge was ensuring the backend smoothly handled situations where users input incomplete or improperly formatted JSON data. This was resolved by validating and displaying clear error messages using Python's try-except blocks.

- Careful type handling was necessary to incorporate the machine learning model into the Flask application. In order to prevent serialization issues while returning JSON replies, predictions that were returned as NumPy types had to be converted into native Python types (int). In the absence of this conversion, the API would respond to users with type errors.

- Although an 80:20 ratio was used to split the dataset into training and testing subsets, this study did not use additional validation methods like k-fold cross-validation. The lack of a separate validation set and the restricted hyperparameter tuning could have affected the model evaluation's reliability. Although in-depth analyses like learning curves or validation loss monitoring were not carried out, overfitting and underfitting were mainly evaluated by comparing the training and testing accuracy scores. To increase reliability in the model's applicability, future research should incorporate more thorough validation techniques and real-world testing.

4.2.2 Schedule Generation

Students' weekly study schedules are automatically generated by this feature based on two primary factors:

- Number of available study hours
- Difficulty level of the subjects

By assigning more study hours to difficult subjects and few hours to easy subjects, this feature makes sure that students make the best use of their time.

```
@app.route('/generate-schedule', methods=['POST'])
def generate_schedule():
    data = request.get_json()

    available_hours = data.get('available_hours')
    subject_difficulty = data.get('subject_difficulty')

    if not available_hours or not subject_difficulty:
        return jsonify({'error': 'Missing data for schedule generation'}), 400

    schedule = generate_schedule(available_hours, subject_difficulty)
    return jsonify({'schedule': schedule})
```

Figure 4.2.2.1 Schedule generation (Client side)

The initial creation of a study schedule is handled by this section of the code.
It gets a POST request with the following information:

- 'available_hours': how much time a student can allocate for studying
- 'subject_difficulty': a list of subjects with their difficulty levels

The input is validated by the function. If either of these values is absent, a 400 error is returned. In order to create a customized schedule, it passes the valid inputs to the internal 'generate_schedule()' method (specified elsewhere). The resulting schedule is returned in JSON format.

```python
def generate_schedule():
    data = request.get_json()
    user_id = data['user_id']
    schedule = data['schedule']

    if not user_id or not schedule:
        return jsonify({'error': 'User ID and subjects are required'}), 400

    try:
        cursor = mysql.connection.cursor()

        cursor.execute("DELETE FROM schedules WHERE user_id = %s", (user_id,))

        days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
        for i, task in enumerate(schedule):
            day = days[i % 7]
            cursor.execute('''
                INSERT INTO schedules (user_id, day, subject, hours)
                VALUES (%s, %s, %s, %s)
            ''', (user_id, day, task['subject'], task['hours']))

        mysql.connection.commit()
        return jsonify({'message': 'Schedule generated and saved successfully'}), 200

    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

Figure 4.2.2.2 Schedule saving logic (Server side)

This function retrieves the generated schedule and stores it in the MySQL database, allocating tasks throughout the week (Monday to Sunday).

It expects:

- ‘user_id’: to identify each user
- ‘schedule’: the schedule generated in the previous step

It first clears the user’s existing schedules to prevent duplication. Then it adds new records by allocating tasks evenly throughout the week. It returns a success message and commits all changes if everything goes well. If something goes wrong, it smoothly handles the exceptions with appropriate error messages.

Challenges faced:

- It was challenging to make sure that tasks were divided equally throughout the week. For example, if a student had 10 tasks, to efficiently map 10 tasks into 7 days, it required modular arithmetic (‘i%7’) to loop through the days without repetition or bias.

- It was essential to validate the schedule structure and user input. Malformed or missing data may cause database issues or interfere with the creation of the schedule. In order to identify problems early, proper input checks were introduced. (‘if not user_id or not schedule’).

- Users might often create new schedules, and if it is not properly cleaned up, the database would fill with duplicate entries. Database hygiene was preserved by deleting existing entries before adding new ones (‘DELETE FROM schedules WHERE user_id=%s’).

4.2.3 User Authentication

This feature ensures that users can register, securely log in, and safely log out of the app. It is a fundamental component of the systems that offer session management and access control for customized services.

```python
@app.route('/signup', methods=['POST'])
def signup():
    data = request.get_json()
    name = data.get('name')
    email = data.get('email')
    password = data.get('password')

    if not all([name, email, password]):
        return jsonify({'error': 'Missing fields'}), 400

    hashed_password = generate_password_hash(password)

    try:
        cur = mysql.connection.cursor()
        cur.execute("INSERT INTO users (name, email, password) VALUES (%s, %s, %s)",
                    (name, email, hashed_password))
        mysql.connection.commit()
        return jsonify({'message': 'User registered successfully'}), 201
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

Figure 4.2.3.1 User signup

It first accepts a POST request that includes name, email, and password. It verifies that all fields are filled; if not, return a 400 error message. To improve the security, it hashes the password using ‘generate_password_hash()’. Then it inserts these details into the users table in the MySQL database. And at the end, if the user is successfully registered, it returns a success message with a 201 status code.

```dart
register_screen.dart ×
lib > register_screen.dart
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'theme.dart';
import 'login_screen.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({Key? key}) : super(key: key);

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();

  final _fullNameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmPasswordController = TextEditingController();

  @override
  void dispose() {
    _fullNameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmPasswordController.dispose();
    super.dispose();
  }
```

Figure 4.2.3.2 Register Screen Flutter Code

```python
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')

    if not all([email, password]):
        return jsonify({"error": "Missing credentials"}), 400

    cur = mysql.connection.cursor()
    cur.execute("SELECT id, password FROM users WHERE email = %s", (email,))
    user = cur.fetchone()

    if user and check_password_hash(user[1], password):
        session['user_id'] = user[0]
        return jsonify({"message": "Login successful", "user_id": user[0]}), 200
    else:
        return jsonify({"error": "Invalid credentials"}), 401

@app.route('/logout', methods=['POST'])
def logout():
    session.clear()
    return jsonify({"message": "Logout successful"}), 200
```

Figure 4.2.3.3 User login and logout

Login

It uses a POST request to accept the email and password. Then it retrieves the corresponding user from the database. It verifies the input password using 'check_password_hash()'. If it is successful, it stores the user's ID in the Flask session. Then returns a 200 status along with the success message and the user ID. If there's a failure, it returns a 401 for invalid credentials.

Logout

In this, it clears the session, effectively logging out the user. It provides a logout confirmation message along with a 200 status.

register_screen.dart login_screen.dart ×

b > login_screen.dart

```dart
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

import 'theme.dart';
import 'register_screen.dart';
import 'dashboard_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({Key? key}) : super(key: key);

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

```

Figure 4.2.3.4 Login Form Layout

Challenges faced:

- In order to switch from plain-text to hashed passwords, ‘werkzeug.security’ functions needed to be learned and tested.
- For consistency and route protection, session data needed to be checked and stored correctly.
- Failure causes were not clearly defined in the initial implementations.

4.3 Code Structure and Organization

To provide a modular and maintainable architecture, the codebase was arranged into the following directories:

- ‘app/’: Contains the main application logic, such as controllers and routes for various functions, including authentication, performance prediction, and scheduling.

- ‘models/’: Holds the database interaction logic and machine learning model.

- ‘utils/’: Contains utility functions like input validation, session handling, and password hashing to encourage code reuse and clear routing.

The backend follows a modular design, with distinct route functions acting as controllers for each functionality. These controllers oversee the logic that responds to JSON requests, handles HTTP requests, and communicates with the database.

A pseudo-MVC methodology was used to capitalize on Flask’s simplicity, where:
- ML models and SQL queries are used to define ML models.
- For API use, JSON replies are used instead of views.
- Controllers are the route functions that connect everything.

Common operations like database connectivity, data validation, and error handling were performed by reusable components. This modularity helps to debug, expand, or refactor particular features without affecting others.

Coding practices include the use of naming conventions (e.g., generate_schedule, predict_performance) that clearly reflect the function’s purpose. Routes adhere to RESTful principles (/login, /signup), and for security and resilience, all sensitive activities make use of appropriate exception handling and input checks.

4.4 Challenges and Solutions

Managing API requests was challenging due to missing or inaccurate JSON data from the Flutter frontend, which caused server crashes. I resolved this by using Flask's 'jsonify()' and thorough error checks to ensure graceful responses with proper HTTP status codes and clear messages.

Handling sessions and authentication was also difficult since the Flutter frontend and Flask backend required careful session management. I initially used Flask's built-in sessions and later added security measures to protect user data and restrict unauthorized access.

Device compatibility was another constraint, as Flutter doesn't support building iOS apps on Windows, preventing testing on real iPhones. To work around this, I relied on online previews and Android emulators to ensure consistent behavior across platforms.

These challenges highlighted the importance of secure sessions, robust error handling, and realistic testing environments.

5. Testing

Testing was a crucial part to ensure the application worked as expected and provided a smooth user experience. This section outlines the test cases and results, the approach, and the tools used in the testing process.

5.1 Overview of Testing Approach

The testing methodology guaranteed the system's functionality, usability and reliability. It was planned to cover both functional and non-functional requirements using techniques like white box testing, black box testing, unit testing and system testing.

Black box testing was used across multiple modules without looking at internal code to confirm user interactions and app behavior. White box testing was used on backend functions to analyze internal logic. Unit tests were run on some essential backend functions such as the schedule generation APIs and the prediction logic to ensure the accuracy of outputs. System testing ensured that the Flask backend, Flutter frontend and MySQL database all integrated seamlessly.

Given the system's intended deployment on Android, testing was done on both simulators and real smartphones running Android 10 and above. To guarantee responsiveness and reliable operation, the application was manually tested on a range of screen sizes and device configurations.

Postman was used for backend API testing to verify expected responses and error handling, while manual testing concentrated on user interface validation and navigation flows.

5.2 Test Plan and Test Cases

Test cases were designed to validate key components such as user login and registration, prediction and schedule generation, and database operations. To guarantee traceability, transparency and reliability, each test case adhered to a structured format. Both successful and unsuccessful scenarios were covered by the manual execution of the tests on emulators and real devices.

5.2.1 Login Module

Test Case ID	**Functionality Tested**	**Input**	**Expected Output**	**Actual Output**	**Pass/Fail**
TC01	Login with valid email and password	Email : ann@gmail.com Password : ann123	Redirect to dashboard	Redirect to dashboard	Pass
TC02	Login with invalid email	Email : ann1@gmail.com Password : ann123	Show error message	Show error message	Pass
TC03	Login with invalid password	Email : ann@gmail.com Password : ann1123	Show error message	Show error message	Pass
TC04	Empty fields	Email : (blank) Password : (blank)	Show “All fields required” message	Show “All fields required” message	Pass

Table 5.1 Login Module Test Cases

5.2.2 Registration Module

Test Case ID	Functionality Tested	Input	Expected Output	Actual Output	Pass/Fail
TC05	Successful registration	Name : Ann Email : ann@gmail.com Password : ann123	Account created and redirect to login	Account created and redirect to login	Pass
TC06	Duplicate emails (Account already created)	Email : ann@gmail.com	Show “Email already registered”	Show “Email already registered”	Pass
TC07	Missing required fields	Name : (blank) Email : ann@gmail.com Password : ann1123	Show “All fields required” message	Show “All fields required” message	Pass
TC08	Weak Password	Email : ann@gmail.com Password : 123	Show “Password too weak” message	Show “Password too weak” message	Pass

Table 5.2 Registration Module Test Cases

5.2.3 Prediction and Schedule Generation Module

Test Case ID	Functionality Tested	Input	Expected Output	Actual Output	Pass/Fail
TC09	Generate prediction	Historical academic data	Display predicted performance	Display predicted performance	Pass
TC10	Generate study schedule	Performance prediction = low	Generate customized study schedule	Generate customized study schedule	Pass

Table 5.3 Prediction and Schedule Generation Module Test Cases

5.2.4 Database Operations

Test Case ID	Functionality Tested	Input	Expected Output	Actual Output	Pass/Fail
TC11	Save user details	Name, email, password	User record saved in MySQL database	User record saved successfully	Pass
TC12	Retrieve study schedule	User ID	Return correct schedule for relevant user ID	Correct schedule returned	Pass

Table 5.4 Database Operations Test Cases

5.3 User Acceptance Testing (UAT)

To assess the real-world usability of the application, a User Acceptance Testing(UAT) was conducted involving both technical and non-technical users who matched the end-user profile of the system. This UAT survey got a total of 20 participants who reflected a range of digital literacy levels and different app usage patterns. The survey was conducted using a Google Form, and the full questionnaire is included in **Appendix B**.

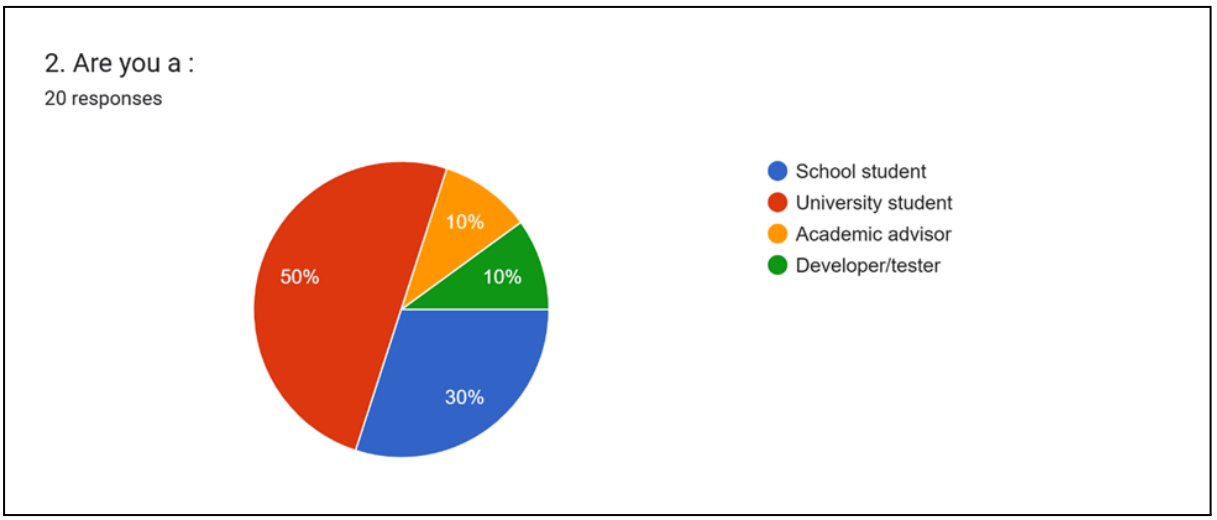


Figure 5.3.1 User Demographics

The user demographics chart illustrates the distribution of participants involved in the User Acceptance Testing. University students(50%) made up the majority of the 20 respondents, followed by school students(30%). Academic advisors and developers/testers each accounted for 10% of the remaining respondants. This diversified group ensured that the end users' and technical reviewers' viewpoints were represented in the feedback. The knowledge acquired was helpful in confirming the system's efficiency, relevance, and usability for various user types.

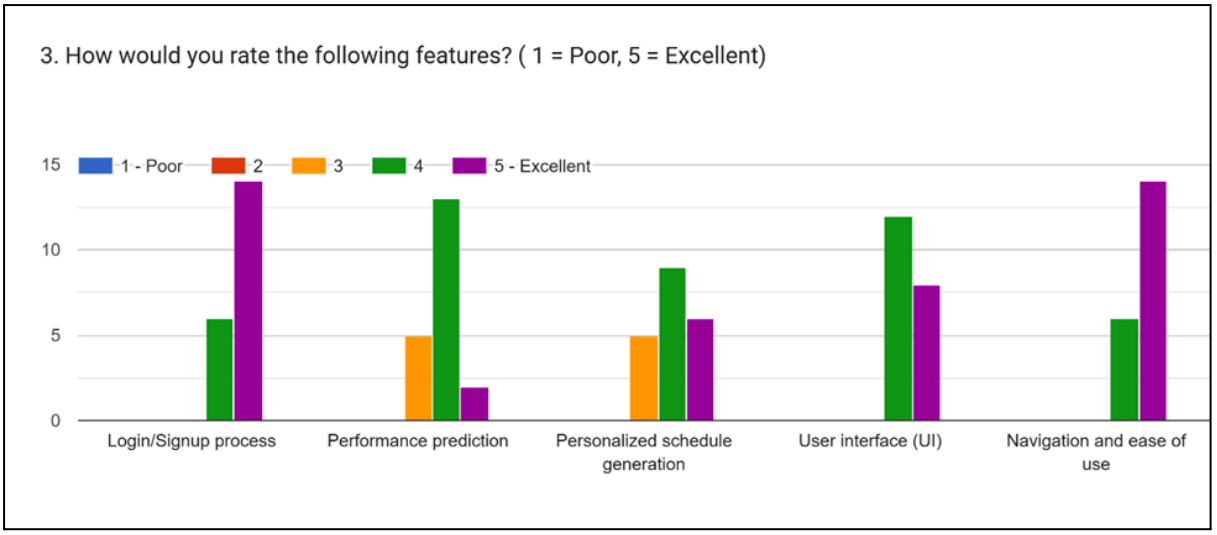


Figure 5.3.2 User Feedback on App Features

On a scale of 1 (Poor) to 5 (Excellent), the bar chart represents user feedback on five key system features. High user satisfaction was indicated by the majority of users' excellent(5) ratings for the navigation and ease of use, and login/signup process.

The majority of ratings for the performance prediction feature was 4, indicating high reliability with only minor potential for improvement. Feedback on personalized schedule generation was more uncertain, with the most common ratings being moderate (3 and 4).

The majority of users provided the User Interface(UI) a rating of 4, and some gave it a 5. Overall, the survey results indicate that users found the system performed well and was easy to use. However, making the personalization features better might enhance the experience even better.

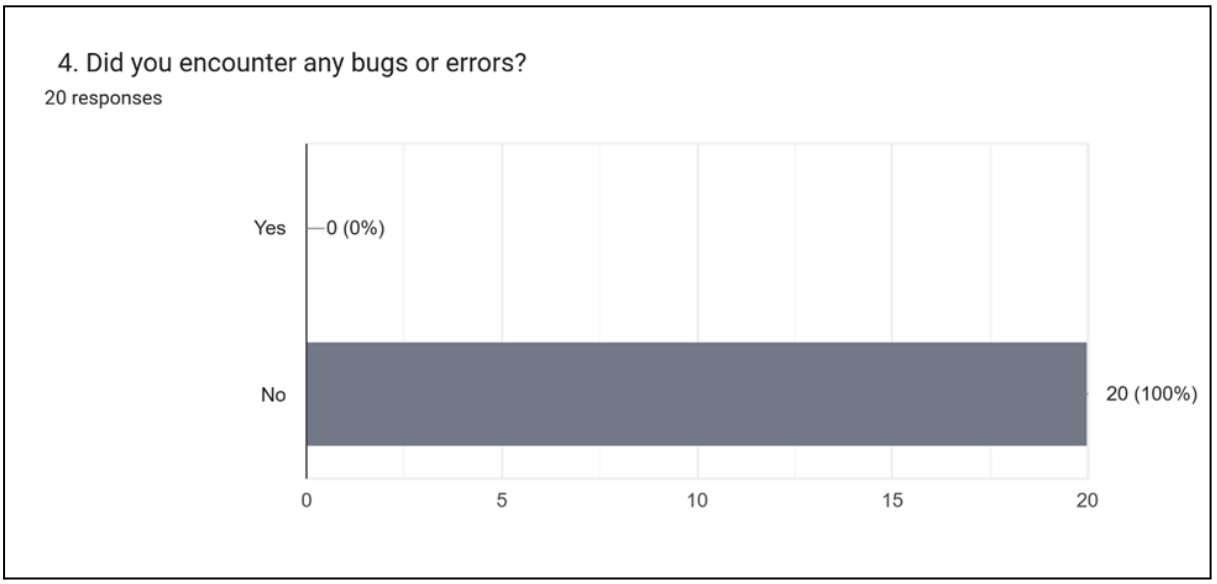


Figure 5.3.3 User Feedback on Bugs

In response to a separate question about bugs, all 20 users had said they did not encounter any bugs during testing. This indicates that the system is technically stable and can handle errors well during the user acceptance testing phase.

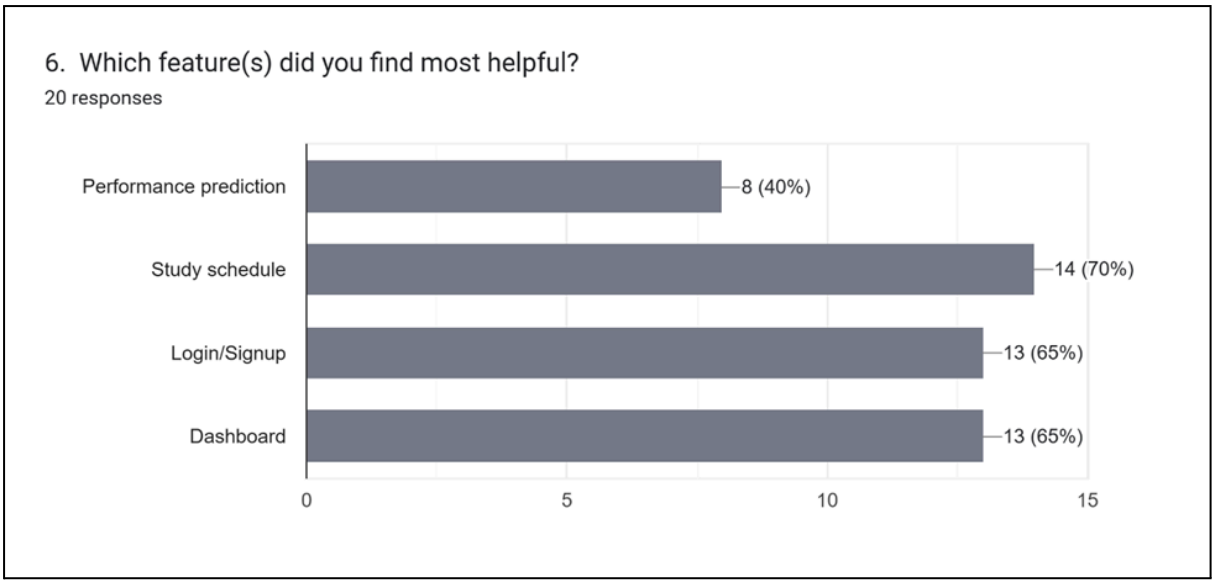


Figure 5.3.4 User Feedback on Most Helpful App Feature(s)

Based on users' responses, the Study Schedule was the most helpful feature, with 70% of respondents choosing it. This indicates that users really liked how the system could generate personalized and organized study plans. The Login/Signup and Dashboard features came in a close second, with 65% of respondents indicating they were helpful. This shows that easy access and a clear overview of information were important for improving the user experience. 40% of users chose the Performance Prediction feature, which suggests that they liked it but that they were more interested in the practical tools that helped them manage their schoolwork. Overall, the results show how important it is for academic support systems to have features that are easy to use and encourage action.

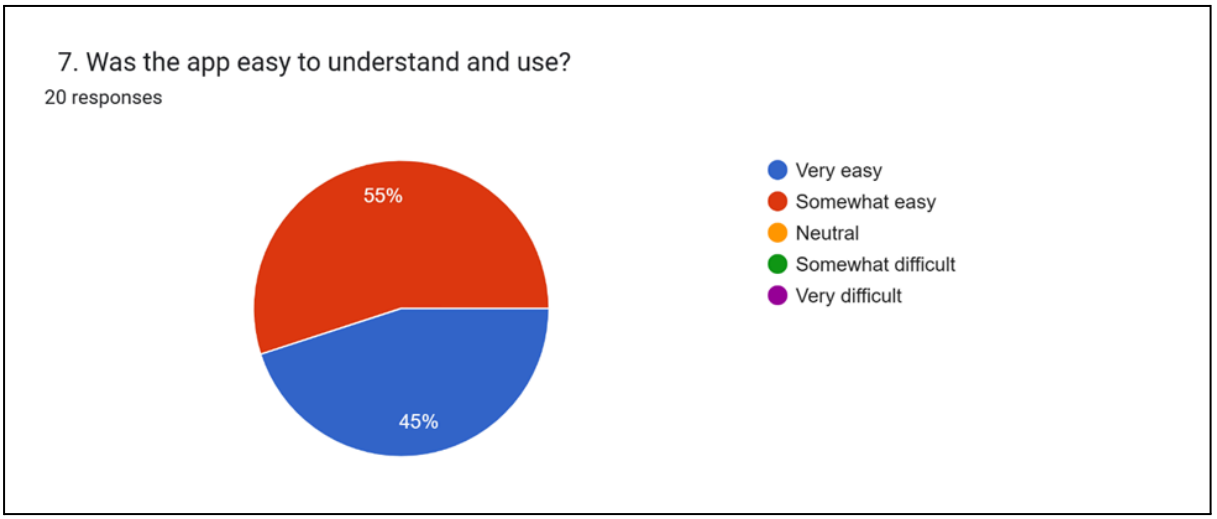


Figure 5.3.5 User Feedback on App Navigation and Usage

When asked how easy it was to use and understand the app, 55% of users said it was "somewhat easy" to navigate, and 45% said it was "very easy" to use. This indicates that most people who took part were able to use the system with little trouble. However, there is still room for improvement in terms of making it more understandable and user-friendly so that more people can move up to the "very easy" category.

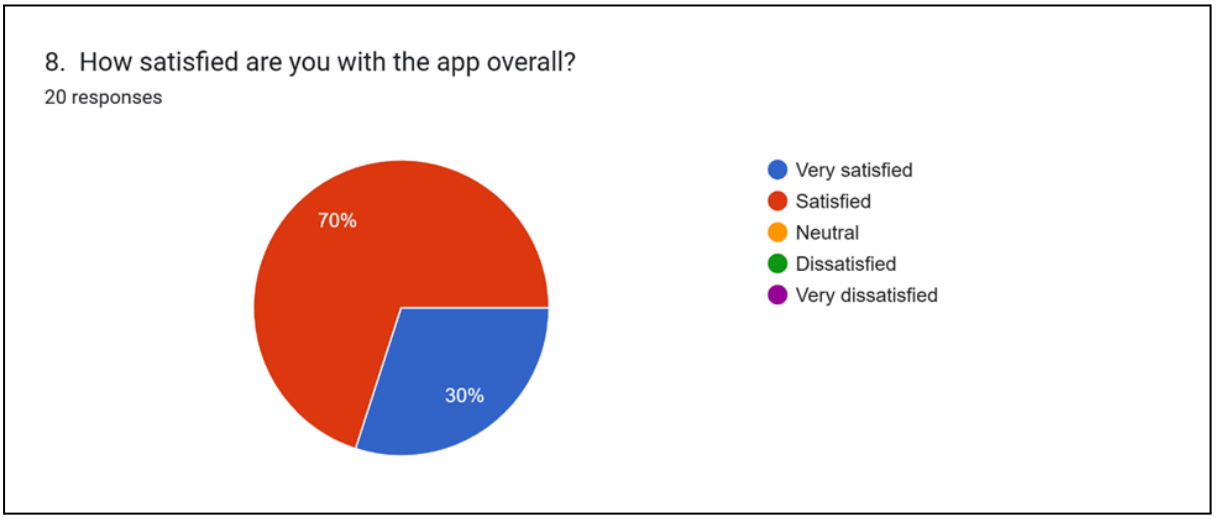


Figure 5.3.6 User Feedback on Satisfaction

Regarding overall satisfaction with the app, 30% of participants said they were extremely satisfied with the app, and 70% said they were satisfied. Although more improvements could enhance the experience and raise the percentage of users who are extremely satisfied, this indicates that most users had a good experience with the application.

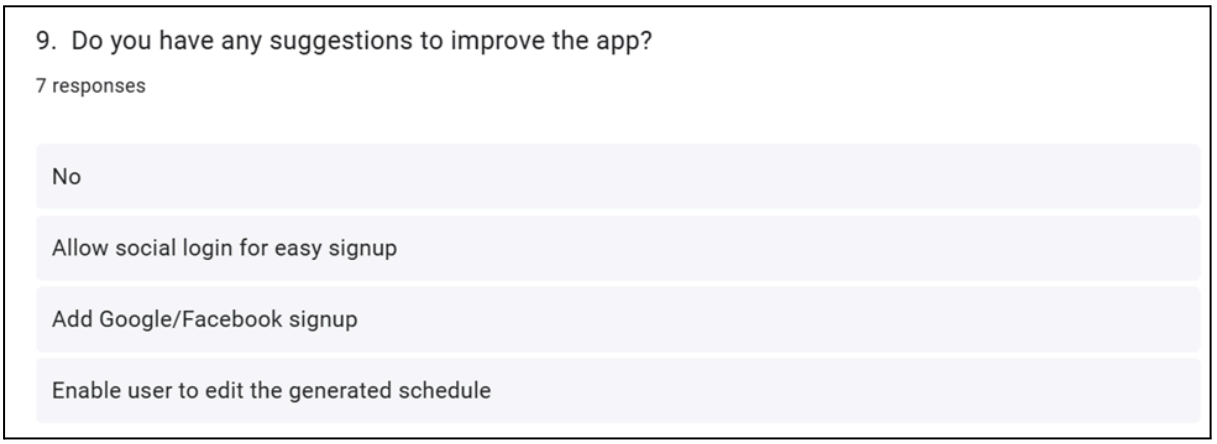


Figure 5.3.7 Improvement Suggestions

Overall satisfaction with the app's functionality was indicated by the fact that 4 out of 20 participants said they had no further suggestions for improvement. Three users did however suggest some specific improvements, such as allowing users to modify their generated schedules and integrating social login options like Google or Facebook for a more smooth signup process. These recommendations are helpful ideas for upcoming system iterations and point towards expanding user flexibility and convenience.

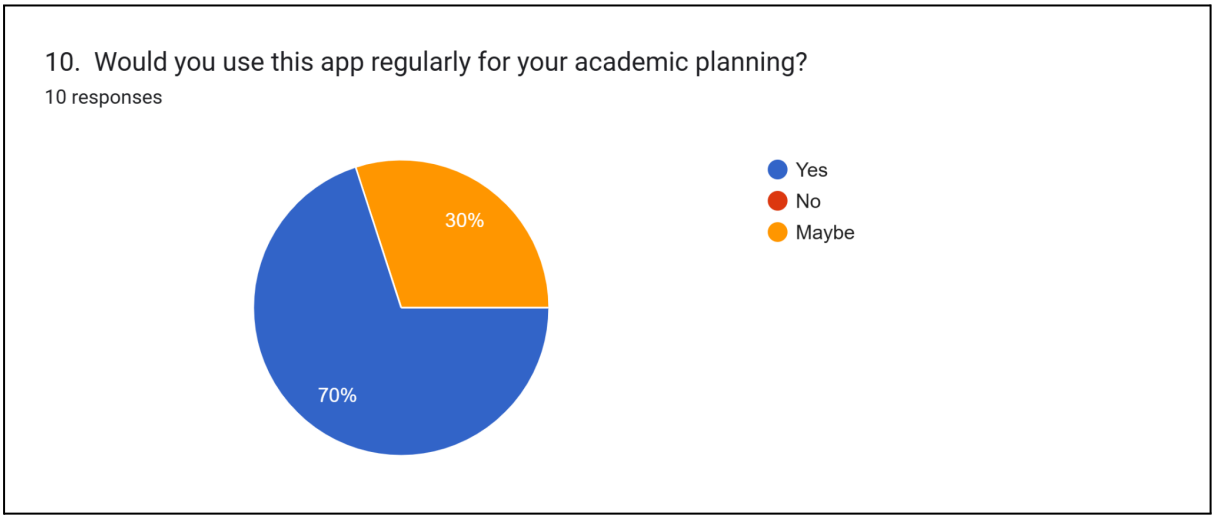


Figure 5.3.8 User Feedback on Future App Usage

70% of participants said they would use the app on a regular basis for academic planning, indicating their strong desire to incorporate the system into their daily routine. The remaining 30% chose "Maybe," indicating the possibility of frequent use in a scenario that specific features or enhancements are added. Overall opinions regarding the app's value and applicability for academic support are reflected in these reviews.

Overall the results of the user acceptance testing were positive. The study schedule and dashboard were rated as the most useful features, and none of the 20 participants complained about any bugs. The majority of users expressed satisfaction and reflected the app had been easy to use. Some recommended improvements entail the ability to edit schedules and enable social login.

5.4 Bug Log and Resolution

To improve system stability and user experience, a number of bugs were found and fixed during development and testing. UI misalignments, validation errors, and logical mistakes in scheduling, prediction, and authentication were frequent problems. Iterative testing and tracking were used to prioritize bugs based on their severity.

A few representative bugs found and their fixes are highlighted in the table below.

Bug ID	**Description**	**Severity**	**Status**	**Resolution**
B01	Signup form allowed empty email input	HIgh	Resolved	Added form validation to ensure required fields are not empty
B02	Schedule page did not display properly on smaller screens	Medium	Resolved	Applied responsive CSS fixes to adjust layout for different screen sizes
B03	User received duplicate notifications for same reminder	Medium	Resolved	Modified backend logic to check for existing reminders before sending
B04	Login failed after password reset	Low	Resolved	Synced frontend form with updated password hashing
B05	“Forgot Password” feature not working due to missing backend	High	Rejected	Feature out of scope for this version. Planned for future implementation

Table 5.5 Bugs and their Resolutions

5.5 Validation

System validation ensured that all the functional and non-functional requirements were met. Key techniques included a requirement traceability matrix, user walkthroughs, and comparison with original specifications.

Every feature, including edge cases like login failures and prediction errors, was tested to ensure it met the requirements. Usability and accessibility were assessed with the assistance of real users.

Non-functional elements like security (e.g., encryption and authentication), performance, and the clarity of the user interface were also checked. The traceability matrix that follows details how each requirement was met, demonstrating that the system goals were fully validated.

Requirement ID	**Description**	**Validated Feature**	**Status**
R01	User account management	Signup/Login module	Validated
R02	Enter academic details	Academic input form	Validated
R03	Personalized academic support	Performance prediction	Validated
R04	Study reminders & notifications	Notification module	Validated
R05	Notify performance decline	Performance drop alerts	Validated
R06	Display predicted analysis	Dashboard analytics	Validated
R07	Generate customized study plans	Schedule generation module	Validated
R08	Analyze historical data	Machine learning backend	Validated
R09	Ensure privacy compliance	Encrypted DB & role-based access	Validated
R10	Respond within 2-5 seconds	Performance test results	Validated
R11	User-friendly interface	Simple UI	Validated
R12	Secure storage	Data encryption	Validated

Table 5.6 Requirements Traceability Matrix

5.6 Summary

In conclusion, testing confirmed that the system is functioning properly and satisfied the core requirements listed in Section 2. The majority of the key features, such as user account management, prediction and scheduling had been successfully validated. Several problems including incorrect notification triggers, login failures and minor UI inconsistencies were found and fixed with the aid of the iterative testing process.

Feedbacks from walkthroughs and functional test cases (see Tables 5.1, 5.2, 5.3, 5.4) verified that user expectations were fulfilled. Performance and data security were some of the non-functional requirements that were addressed and partially validated. Through retesting and logic enhancements, critical bugs pertaining to schedule updates and performance prediction were fixed.

The system has been determined stable and deployment-ready based on the test results and requirements traceability (see Table 5.6). Future iterations may have better performance and scalability through additional improvements.

6. Critical Appraisal

This section critically evaluates how well the initial goals were achieved, the effectiveness of the methodologies utilized, and the challenges faced during the process. With this evaluation, I intend to assess the project's strengths and weaknesses and identify important takeaways for future enhancements.

6.1 Achievements Against Objectives

The proposal's primary goals were to use historical data to predict student performance, generate customized study plans, incorporate a machine learning model into a workable system, and make sure that students could easily navigate the interface.

In line with my primary objective, I implemented the key functionality for performance prediction using historical data. Furthermore, the planned development of a customized scheduling system had been accomplished and during testing, this feature reacted favorably to user input. Additionally, I was able to create a seamless connection between the mobile application and the backend services, which enhanced user experience and allowed for real-time interaction.

While I am satisfied with how authentication and UI features were implemented, I admit that one goal, improving academic engagement by sending out reminders or notifications, was not met. During development, I ran into issues with time and prioritization, which caused me to postpone this feature.

Below is a comparison of the goals that were planned and those that were accomplished.

Objective	**Status**	**Evidence**
Predict student performance using Machine Learning	Fully achieved	Prediction API tested with 99% accuracy
Generate personalized study schedules	Fully achieved	Schedule API functional and tested
Integrate backend with frontend for real-time access	Fully achieved	API integration verified in app
Provide user-friendly interface and authentication	Fully achieved	Feedback from User Acceptance Testing
Improve academic engagement through reminders	Not achieved	Notification module not implemented

Table 6.1 Comparison of planned goals and accomplished goals

This analysis demonstrates that, despite one shortcoming, the project's main objectives were met and closely aligned with the initial goals.

6.2 Challenges Faced

Throughout the development lifecycle, a number of challenges were faced, both technical and non-technical. Effective time management in addition to academic obligations was one of the main challenges which resulted in delays during important stages of development, especially backend integration and testing.

In terms of technology, it took a lot of time to learn and apply new technologies like the Flask framework and integrate it with the Flutter frontend. Comprehensive research and troubleshooting were necessary for the integration of database handling and APIs, particularly when unexpected problems arose during deployment.

Stakeholder interaction caused some more non-technical difficulties. Regular follow-ups were necessary to clarify expectations and guarantee consistent feedback, but conflicting schedules at times made this unattainable.

Through early planning and self-learning, I was able to overcome the majority of these challenges. However, time constraints forced the simplification of some features, like advanced notification scheduling. All things considered, the challenges significantly improved my potential for learning and adaptability.

6.3 Lessons Learned

This project provided valuable expertise in managing every phase of the software development cycle in the real world, from planning to implementation. The value of structured project management was among the most significant lessons that I learned. I was more able to monitor my progress and respond to setbacks when I split the development process into smaller milestones.

I gained a deeper understanding of how various technologies work together in practice by working on both frontend and backend components. I was able to observe how theoretical knowledge is converted into working systems by implementing REST APIs into practice, connecting them to a MySQL database, and incorporating the machine learning model. It also emphasized how crucial iterative testing and debugging are to maintaining functionality and stability.

The significance of stakeholder communication was another important lesson. I discovered that even the most technically sound features need to be designed with the end user in mind and presented in an understandable manner.

The experience also improved my capacity for self-learning and adaptation. I had the opportunity to explore a number of tools and frameworks, which enhanced my technical confidence and research abilities.

Additionally, I realized that most of the code used to develop the system was made up of boilerplate standard components, especially in the Flutter UI and the basic Flask API endpoints. The most significant and unique contributions were made in integrating the machine learning model, managing prediction requests, and implementing solid input validation into practice, even though these were necessary to develop a working end-to-end system. I gained a better understanding of the connection between developing unique logic to address particular issues and efficiently utilizing pre-existing frameworks.

Overall, the project improved my knowledge of professional communication, user-centered design, and full-stack development, skills that are essential in any future position involving software engineering.

6.4 Future Improvements

Several important features identified during the user acceptance testing phase could be incorporated to the current system to improve it. Adding social login options, like Google or Facebook, would be a significant improvement as it would speed up the signup process and enhance user convenience. The flexibility and usability of the system would also be improved by allowing users to modify the study schedules that are generated, according to user feedback.

Even though no issues were discovered in the initial tests, the incorporation of advanced testing tools in later versions may help guarantee greater reliability and early bug detection. A more scalable backend architecture could assist in helping the system manage more users as popularity increases. A greater user experience might also be provided by adding progress tracking or personalized insights to the dashboard.

Coordinating thousands of users' requests at once, guaranteeing low prediction delays, and implementing load balancing in place for heavy traffic are some potential future scalability issues.

All things considered, these enhancements motivated by user input and scalability objectives would result in a more advanced and user-centric system.

6.5 Reflection on Development Process

The use of Agile methodology allowed iterative development, which helped with task management and change response based on ongoing feedback. It was simpler to prioritize features, monitor progress, and test individual parts without being overwhelmed when the project was divided into smaller sprints. Maintaining focus and gradually improving the system were also aided by regular evaluations following each phase.

Ethical standards were adhered by complying with data privacy guidelines during data storage and transfer. In education, predictive systems run the risk of bias or unfairly classifying students. In order to fix this, the model's attributes were examined to make sure that no explicitly discriminatory variables (gender, race) were used.

If I were to repeat this project, I would consider spending more time on initial planning and detailed requirement collection. With every aspect considered, the development process showed me the importance of adaptability, and customer feedback in creating a system that is both helpful and simple to use.

6.5.1 Reflection on Machine Learning Techniques and Data Limitations

Three machine learning algorithms were assessed for their ability to predict student academic performance in this project: Decision Tree, Support Vector Machine (SVM), and Logistic Regression. With an accuracy of 99%, backed by a similarly high F1-score of 0.99, the Decision Tree model outperformed both SVM and Logistic Regression, which both had an accuracy of 86%. Because of its exceptional predictive capabilities and interpretability, which provides insight into the decision-making process, the Decision Tree model was selected as the final model.

These algorithms were selected based on their suitability for classification tasks and their range of complexity. SVM is known for its efficient performance on high-dimensional spaces, Decision Trees provide an easy-to-understand visual representation of decision rules, which is useful in educational settings, and Logistic Regression offers a baseline linear model.

However, there are some restrictions when using a Kaggle dataset that is openly accessible. Public datasets might not fully reflect the range of student populations or learning environments, have sampling biases, and have limited feature diversity. Practical elements that have a major influence on student performance, like curriculum variations, financial status, and the level of guidance, may be lacking in this dataset.

As a result, even though the Decision Tree model performs exceptionally well on the test data, it is unclear if it will apply to other student populations. Applying the model to data from other locations or periods may cause its performance to deteriorate. Future research could involve gathering more current and varied data, investigating ensemble methods to increase generalizability and reliability, and feature engineering to capture more contextual factors.

6.5.2 Code Origin and Adaptation Declaration

This section indicates which portions of the code are boilerplate, original, and modified from open sources in order to maintain transparency.

Module	Code Origin
Flutter UI Screens	Mostly boilerplate modified from Flutter templates and documents
Flask API Endpoints	Boilerplate modified with a few minor modifications
ML Model Training	For this project, entirely unique logic was created
Prediction API Logic	Original logic with input validation adapted from examples
Error Handling	Initial implementation based on Python best practices

Table 6.2 Code Origin and Adaptation

7. References

Student Exam Performance Prediction. 2024. Available at: https://www.kaggle.com/datasets/mrsimple07/student-exam-performance-prediction [Accessed: 20 April 2025].

Moodle. (2025) About Moodle. Available at: https://moodle.com/about [Accessed: 8 April 2025].

Khan Academy. (2025) About. Available at: https://www.khanacademy.org/about [Accessed: 8 April 2025].

OpenAI. (2025) OpenAI Academy. Available at: https://openai.com/openai-academy [Accessed: 8 April 2025].

Django Software Foundation. (2025) The Web framework for perfectionists with deadlines. Available at: https://www.djangoproject.com [Accessed: 8 April 2025].

Scikit-learn. (2025) Machine Learning in Python. Available at: https://scikit-learn.org [Accessed: 8 April 2025].

Flutter. (2025) Build apps for any screen. Available at: https://flutter.dev [Accessed: 8 April 2025].

draw.io - free flowchart maker and diagrams online (no date). Available at: https://app.diagrams.net/ [Accessed: 26 April 2025].

Figma - The Collaborative Interface Design Tool (no date). Available at: https://www.figma.com/ [Accessed: May 1, 2025].

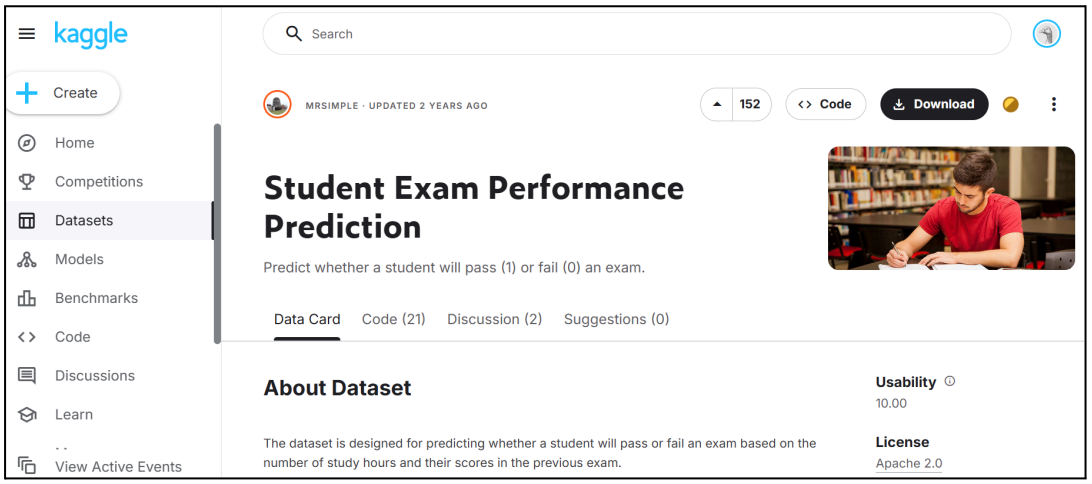
Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html [Accessed: August 2, 2025].

Oracle. (2025) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/ [Accessed: August 3, 2025].

8. Appendices

8.1 Appendix A: Source Files

Kaggle Student Performance Dataset:
https://www.kaggle.com/datasets/mrsimple07/student-exam-performance-prediction



student_exam_data_new.csv File Link:
https://drive.google.com/file/d/1dRRrrLPkHcLbSx-v3eDvIv-rNoUAK5fK/view?usp=sharing

The complete project source code and supporting files are provided in here:
https://drive.google.com/drive/folders/1U1KmPcb4cEH-Vz_ciGQHBJ6GJdwFchaR?usp=sharing

8.2 Appendix B: Source Codes

B.1 main.dart

```
import 'package:flutter/material.dart';
import 'theme.dart';
import 'home_screen.dart';

void main() => runApp(const SmartStudyApp());

class SmartStudyApp extends StatelessWidget {
  const SmartStudyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      title: 'SmartStudy',
      theme: ThemeData(
        primaryColor: AppColors.blue,
        scaffoldBackgroundColor: AppColors.lightBlue,
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: AppColors.buttonBlue,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(10),
            ),
          ),
        ),
      ),
      home: const HomeScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}
```

B.2 theme.dart

```
import 'package:flutter/material.dart';

class AppColors {
  static const Color lightBlue = Color(0xFFD4EBFD);
  static const Color blue = Color(0xFF8CA8D4);
  static const Color buttonBlue = Color(0xFF4D648D);
  static const Color highlightBlue = Color(0xFFAED2F2);
  static const Color darkBlue = Color(0xFF203864);
}
```

B.3 register_screen.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'theme.dart';
import 'login_screen.dart';

class RegisterScreen extends StatefulWidget {
```

```
  const RegisterScreen({Key? key}) : super(key: key);

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();

  final _fullNameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmPasswordController = TextEditingController();

  @override
  void dispose() {
    _fullNameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmPasswordController.dispose();
    super.dispose();
  }

  Future<void> _register() async {
    if (!_formKey.currentState!.validate()) return;

    final uri = Uri.parse('http://10.0.2.2:5000/register');

    try {
      final response = await http.post(
        uri,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'name': _fullNameController.text.trim(),
          'email': _emailController.text.trim(),
          'password': _passwordController.text.trim(),
        }),
      );

      if (response.statusCode == 200) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Registration successful!')),
        );
```

```
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (_) => const LoginScreen()),
        );
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Registration failed:
${response.body}')),
        );
      }
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Error: $e')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.lightBlue,
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(horizontal: 32),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Create Account',
                  style: TextStyle(
                    fontSize: 28,
                    fontWeight: FontWeight.bold,
                    color: AppColors.darkBlue,
                  ),
                ),
                const SizedBox(height: 24),
                _buildTextField(
                  label: 'Full Name',
                  controller: _fullNameController,
                  validator: (value) {
```

```
                    if (value == null || value.isEmpty) {
                      return 'Please enter your full name';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),
                _buildTextField(
                  label: 'Email',
                  controller: _emailController,
                  keyboardType: TextInputType.emailAddress,
                  validator: (value) {
                    if (value == null || value.isEmpty) {
                      return 'Please enter your email';
                    }
                    if
(!RegExp(r'^[^@]+@[^@]+\.[^@]+').hasMatch(value)) {
                      return 'Please enter a valid email';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),
                _buildTextField(
                  label: 'Password',
                  controller: _passwordController,
                  obscureText: true,
                  validator: (value) {
                    if (value == null || value.isEmpty) {
                      return 'Please enter a password';
                    }
                    if (value.length < 6) {
                      return 'Password must be at least 6 characters';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),
                _buildTextField(
                  label: 'Confirm Password',
                  controller: _confirmPasswordController,
                  obscureText: true,
                  validator: (value) {
```

```
                    if (value != _passwordController.text) {
                      return 'Passwords do not match';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 32),
                SizedBox(
                  width: double.infinity,
                  child: ElevatedButton(
                    style: ElevatedButton.styleFrom(
                      backgroundColor: AppColors.buttonBlue,
                      padding: const EdgeInsets.symmetric(vertical:
16),
                    ),
                    onPressed: _register,
                    child: const Text(
                      'Register',
                      style: TextStyle(fontSize: 16),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                Row(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Text("Already have an account?"),
                    TextButton(
                      onPressed: () {
                        Navigator.pop(context);
                      },
                      child: const Text("Login"),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
```

```
  Widget _buildTextField({
    required String label,
    TextEditingController? controller,
    bool obscureText = false,
    TextInputType keyboardType = TextInputType.text,
    String? Function(String?)? validator,
  }) {
    return TextFormField(
      controller: controller,
      obscureText: obscureText,
      keyboardType: keyboardType,
      validator: validator,
      decoration: InputDecoration(
        labelText: label,
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(borderRadius:
BorderRadius.circular(8)),
      ),
    );
  }
}
```

B.4 login_screen.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

import 'theme.dart';
import 'register_screen.dart';
import 'dashboard_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({Key? key}) : super(key: key);

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _email = TextEditingController();
```

```
  final _password = TextEditingController();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    if (!_formKey.currentState!.validate()) return;

    final uri = Uri.parse('http://10.0.2.2:5000/login');
    try {
      final res = await http.post(
        uri,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'email': _email.text.trim(),
          'password': _password.text.trim(),
        }),
      );

      if (res.statusCode == 200) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(const SnackBar(content: Text('Login success')));

        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (_) => const DashboardScreen()),
        );
      } else {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text('Login failed:
${res.body}')));
      }
    } catch (e) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text('Cannot reach server:
$e')));
```

```
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.lightBlue,
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(horizontal: 32),
          child: Form(
            key: _formKey,
            child: Column(
              children: [
                Image.asset('assets/images/logo.png', height: 120),
                const SizedBox(height: 24),
                Text(
                  'Welcome Back!',
                  style: TextStyle(
                    fontSize: 28,
                    fontWeight: FontWeight.bold,
                    color: AppColors.darkBlue,
                  ),
                ),
                const SizedBox(height: 32),
                _field(
                  label: 'Email',
                  controller: _email,
                  keyboardType: TextInputType.emailAddress,
                  validator: (v) => (v == null || v.isEmpty)
                      ? 'Enter email'
                      : !RegExp(r'^[^@]+@[^@]+\.[^@]+').hasMatch(v)
                      ? 'Invalid email'
                      : null,
                ),
                const SizedBox(height: 16),
                _field(
                  label: 'Password',
                  controller: _password,
                  obscure: true,
                  validator: (v) =>
                      (v == null || v.isEmpty) ? 'Enter password' :
null,
```

```
                ),
                const SizedBox(height: 32),
                SizedBox(
                  width: double.infinity,
                  child: ElevatedButton(
                    onPressed: _login,
                    style: ElevatedButton.styleFrom(
                      backgroundColor: AppColors.buttonBlue,
                      padding: const EdgeInsets.symmetric(vertical:
16),
                    ),
                    child: const Text('Login', style:
TextStyle(fontSize: 16)),
                  ),
                ),
                const SizedBox(height: 16),
                Row(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Text("Don't have an account?"),
                    TextButton(
                      onPressed: () => Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) => const RegisterScreen(),
                        ),
                      ),
                      child: const Text('Register'),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }

  Widget _field({
    required String label,
    required TextEditingController controller,
    bool obscure = false,
```

```
    TextInputType keyboardType = TextInputType.text,
    String? Function(String?)? validator,
  }) {
    return TextFormField(
      controller: controller,
      obscureText: obscure,
      keyboardType: keyboardType,
      validator: validator,
      decoration: InputDecoration(
        labelText: label,
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(borderRadius:
BorderRadius.circular(8)),
      ),
    );
  }
}
```

B.5 home_screen.dart

```
import 'package:flutter/material.dart';
import 'login_screen.dart';
import 'register_screen.dart';
import 'theme.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.lightBlue,
      body: Center(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 32),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Image.asset('assets/images/logo.png', height: 120),
              const SizedBox(height: 24),
```

```
              Text(
                'SmartStudy',
                style: TextStyle(
                  fontSize: 32,
                  fontWeight: FontWeight.bold,
                  color: AppColors.darkBlue,
                ),
              ),
              const SizedBox(height: 8),
              Text(
                'Smart planning for smart results',
                style: TextStyle(fontSize: 16, color:
AppColors.darkBlue),
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 48),
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () => Navigator.push(
                    context,
                    MaterialPageRoute(builder: (_) => const
LoginScreen()),
                  ),
                  child: const Text('Login'),
                ),
              ),
              const SizedBox(height: 16),
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () => Navigator.push(
                    context,
                    MaterialPageRoute(builder: (_) => const
RegisterScreen()),
                  ),
                  child: const Text('Register'),
                ),
              ),
            ],
          ),
        ),
      ),
```

```
    );
  }
}
```

B.6 dashboard_screen.dart

```
import 'package:flutter/material.dart';
import 'theme.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppColors.lightBlue,
      appBar: AppBar(
        title: const Text('Dashboard'),
        backgroundColor: AppColors.buttonBlue,
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: ListView(
          children: [
            _tile(
              icon: Icons.notifications,
              title: 'Performance Alert',
              subtitle: 'You're behind on Chemistry - check smart
schedule.',
            ),
            const Divider(),
            _tile(
              icon: Icons.schedule,
              title: 'Smart Schedule',
              subtitle: '8:00-9:30 AM: Math Revision',
            ),
            const Divider(),
            _tile(
              icon: Icons.bar_chart,
              title: 'Weekly Progress',
              subtitle: '85% target achieved',
```

```
            ),
          ],
        ),
      ),
    );
  }

  Widget _tile({
    required IconData icon,
    required String title,
    required String subtitle,
  }) {
    return ListTile(
      leading: Icon(icon, color: AppColors.buttonBlue),
      title: Text(title, style: TextStyle(fontWeight:
FontWeight.bold)),
      subtitle: Text(subtitle),
    );
  }
}
```

B.7 train_model.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, f1_score
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import numpy as np
import joblib

#Load dataset
data =
pd.read_csv(r'C:\Users\ASUS\Downloads\student_exam_data_new.csv')

X = data[['Study Hours','Previous Exam Score']]  #Prediction features
y = data['Pass/Fail']                            #Target label
```

```
#Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

results = {}

#Train Logistic Regression model
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
log_preds = log_model.predict(X_test)
log_acc = accuracy_score(y_test, log_preds)
results['Logistic Regression'] = (log_acc, log_model)

#Train SVM model
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)
svm_preds = svm_model.predict(X_test)
svm_acc = accuracy_score(y_test, svm_preds)
results['SVM'] = (svm_acc, svm_model)

#Train Decision Tree Classifier model
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X_train, y_train)
tree_preds = tree_model.predict(X_test)
tree_acc = accuracy_score(y_test, tree_preds)
results['Decision Tree'] = (tree_acc, tree_model)

for name, (acc, _) in results.items():
    print(f"{name} Accuracy: {acc:.4f}")

#Identify and save best model based on accuracy
best_model_name = max(results, key=lambda x: results[x][0])
best_model = results[best_model_name][1]
joblib.dump(best_model, 'student_model.pkl')
print(f"\nBest model: {best_model_name} saved as 'student_model.pkl'")

#F1-score and Classifiaction report
y_pred = best_model.predict(X_test)
print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
print(classification_report(y_test, y_pred))

#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```

B.8 app.py

```
import os
import numpy as np
from flask import Flask, request, jsonify, session
from flask_cors import CORS
from flask_session import Session
from flask_mysqldb import MySQL
from werkzeug.security import generate_password_hash,
check_password_hash
import joblib

app = Flask(__name__)
CORS(app, supports_credentials=True)

app.secret_key = 'your_secret_key'
app.config['SESSION_TYPE'] = 'filesystem'
Session(app)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'binki123'
app.config['MYSQL_DB'] = 'student_app'

mysql = MySQL(app)

model =
joblib.load(r'C:\Users\User\AppData\Local\Programs\Python\Python313\stu
dent_performance_app\backend\student_performance_app_backend\model\stud
ent_model.pkl')
print("Model loaded successfully.")

@app.route('/')
def index():
    return 'Flask + MySQL + ML Backend Running!'

@app.route('/register', methods=['POST'])
def register():
```

```
    data = request.get_json()
    name = data.get('name')
    email = data.get('email')
    password = data.get('password')

    if not all([name, email, password]):
        return jsonify({'success': False, 'message': 'Missing
fields'}), 400

    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM users WHERE email = %s", (email,))
    existing_user = cur.fetchone()
    if existing_user:
        return jsonify({'success': False, 'message': 'Email already
exists'}), 409

    hashed_password = generate_password_hash(password)
    try:
        cur.execute("INSERT INTO users (name, email, password) VALUES
(%s, %s, %s)",
                    (name, email, hashed_password))
        mysql.connection.commit()
        return jsonify({'success': True, 'message': 'User registered
successfully'}), 201
    except Exception as e:
        return jsonify({'success': False, 'message': str(e)}), 500

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')

    if not all([email, password]):
        return jsonify({"success": False, "message": "Missing
credentials"}), 400

    cur = mysql.connection.cursor()
    cur.execute("SELECT id, name, email, password FROM users WHERE
email = %s", (email,))
    user = cur.fetchone()

    if user and check_password_hash(user[3], password):
```

```
        session['user_id'] = user[0]
        return jsonify({
            "success": True,
            "message": "Login successful",
            "user": {
                "id": user[0],
                "name": user[1],
                "email": user[2]
            }
        }), 200
    else:
        return jsonify({"success": False, "message": "Invalid
credentials"}), 401

@app.route('/logout', methods=['POST'])
def logout():
    session.clear()
    return jsonify({"message": "Logout successful"}), 200

@app.route('/predict-performance', methods=['POST'])
def predict_performance():
    if 'user_id' not in session:
        return jsonify({"error": "You must be logged in"}), 401

    data = request.get_json()
    try:
        features = [
            data['gender'], data['ethnicity'],
data['parental_level_of_education'],
            data['lunch'], data['test_preparation_course'],
data['math_score'],
            data['reading_score'], data['writing_score']
        ]
        features_array = np.array(features).reshape(1, -1)
        prediction = model.predict(features_array)[0]
        return jsonify({'prediction': int(prediction)})
    except KeyError:
        return jsonify({'error': 'Missing one or more required
fields'}), 400
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/generate-schedule', methods=['POST'])
```

```
def generate_schedule():
    if 'user_id' not in session:
        return jsonify({"error": "You must be logged in"}), 401

    data = request.get_json()
    user_id = session['user_id']
    schedule = data.get('schedule')

    if not schedule:
        return jsonify({'error': 'Schedule data is required'}), 400

    try:
        cursor = mysql.connection.cursor()
        cursor.execute("DELETE FROM schedules WHERE user_id = %s",
(user_id,))

        days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday', 'Sunday']
        for i, task in enumerate(schedule):
            day = days[i % 7]
            cursor.execute('''
                INSERT INTO schedules (user_id, day, subject, hours)
                VALUES (%s, %s, %s, %s)
            ''', (user_id, day, task['subject'], task['hours']))

        mysql.connection.commit()
        return jsonify({'message': 'Schedule generated and saved
successfully'}), 200

    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    print("Registered routes:", app.url_map)
    app.run(debug=True)
```

8.3 Appendix C: Backend/API Evidence

```
{
  "status": "success",
  "study_plan_id": 102,
  "generated_at": "2025-05-12T14:32:11"
}
```

```
{
  "status": "failure",
  "study_plan_id": NULL,
  "generated_at": "2025-05-12T16:30:00",
  "error": "Insufficient data to generate plan"
}
```

```
{
  "status": "success",
  "study_plan_id": 104,
  "generated_at": "2025-05-12T17:45:33"
}
```

8.4 Appendix D: User Impact

User record before study plan generation:

user_id	name	predicted_performance	study_plan	last_updated
101	Alex R.	Low	NULL	2025-08-01 10:00

User record after study plan module updated:

user_id	name	predicted_performance	study_plan	last_updated
101	Alex R.	Low	"Math 2h, Science 1.5h"	2025-08-01 10:05

8.5 Appendix E: Traceability Table

Objective	Report Section	Appendix Section
Performance prediction	4.2.1 Performance Prediction Module	8.1 Appendix A: Source Files B.7 train_model.py
Study plan generation	4.2.2 Schedule Generation	B.8 app.py
User authentication	4.2.3 User Authentication	8.1 Appendix A: Source Files B.3 register_screen.dart B.4 login_screen.dart
Database management	3.4 Database Design	8.4 Appendix D: User Impact

8.6 Appendix F: UAT Feedback Form

SmartStudy App – User Acceptance Testing (UAT) Feedback

This form is part of my BCS Professional Project conducted independently for academic purposes. The goal of this testing session is to assess the real-world usability and functionality of the SmartStudy mobile application, which assists students by predicting academic performance and generating personalized study schedules.

All responses will be treated as strictly confidential and used solely to improve the system as part of the project. Your feedback is highly valuable, and I truly appreciate your time and support in completing this form.

1. Name : *

Your answer

2. Are you a : *

- ◯ School student
- ◯ University student
- ◯ Academic advisor
- ◯ Developer/tester
- ◯ Other: _____

3. How would you rate the following features? (1 = Poor, 5 = Excellent) *

	1 - Poor	2	3	4	5 - Excellent
Login/Signup process	○	○	○	○	○
Performance prediction	○	○	○	○	○
Personalized schedule generation	○	○	○	○	○
User interface (UI)	○	○	○	○	○
Navigation and ease of use	○	○	○	○	○

4. Did you encounter any bugs or errors? *

- [ ] Yes
- [ ] No

5. If Yes, please describe them.

Your answer

6. Which feature(s) did you find most helpful? *

- [ ] Performance prediction
- [ ] Study schedule
- [ ] Login/Signup
- [ ] Dashboard
- [ ] Other: _____

7. Was the app easy to understand and use? *

- ○ Very easy
- ○ Somewhat easy
- ○ Neutral
- ○ Somewhat difficult
- ○ Very difficult

8. How satisfied are you with the app overall? *

- ○ Very satisfied
- ○ Satisfied
- ○ Neutral
- ○ Dissatisfied
- ○ Very dissatisfied

9. Do you have any suggestions to improve the app?

Your answer

10. Would you use this app regularly for your academic planning? *

- ◯ Yes
- ◯ No
- ◯ Maybe

Submit

Clear form

Never submit passwords through Google Forms.